

ANNIE Co-primary μ^- / π^+ Cherenkov Analysis

Geant4 FTFP_BERT | Shuffled 30k sample

Notebook structure:

1. **Imports** — all packages
2. **Load ROOT** — open file, read EventTree + helpers
3. **Definitions** — physics constants, helper functions, signal filter
4. **Computations** — derived columns, masks, StepTree accumulators
5. **Plots** — one cell per figure (labels easy to edit)

Export PNGs are included in each plot cell but **commented out** (`# fig.savefig(...)`).
Un-comment a line to write a 600 dpi PNG next to the ROOT file.

Cell 1 — Imports

```
In [*]: 1 import uproot
        2 import numpy as np
        3 import pandas as pd
        4 import matplotlib.pyplot as plt
        5 import matplotlib.gridspec as gridspec
        6 from pathlib import Path
        7 from IPython.display import display, Markdown
        8
        9 plt.rcParams.update({
10     'figure.dpi': 130,
11     'font.size': 11,
12     'axes.grid': True,
13     'grid.alpha': 0.3,
14 })
15
16 print('Imports OK')
```

Imports OK

Imports OK

Cell 2 — Load ROOT file

In [8]:

```
1 # — Path to ROOT file —————
2 SHUF_PATH = Path(r"D:\research\ANNIE\simualtion\my_geant4\annie_coprimary_
3
4 # PNG export directory – same folder as the ROOT file (un-comment savefig
5 PNG_DIR = SHUF_PATH.parent
6
7 # — Helper: pick tree with most entries —————
8 def choose_best_tree(root_file, tree_name):
9     matches = []
10    for key in root_file.keys():
11        if key.split(";")[0] == tree_name:
12            obj = root_file[key]
13            n = getattr(obj, "num_entries", -1)
14            matches.append((key, n))
15    if not matches:
16        return None
17    matches.sort(key=lambda t: t[1], reverse=True)
18    return root_file[matches[0][0]]
19
20 # — Quick file check —————
21 with uproot.open(SHUF_PATH) as f:
22     print("Keys found:")
23     for k in sorted(f.keys()):
24         print(f" {k}")
25
26 print("ROOT file OK")
```

Keys found:

```
EventTree;1
StepTree;126
StepTree;127
```

ROOT file OK

Cell 3 — Definitions (constants, helpers, Frank-Tamm, signal filter)

In [9]:

```

1  # — Detector geometry —————
2  R_CYL      = 1600.0          # mm, ANNIE inner radius
3  Z_MAX      = 2000.0          # mm, half-height
4  DET_EFF     = 0.025          # PMT geometric efficiency
5  C_LIGHT    = 299.792        # mm/ns
6  N_WATER    = 1.34
7  C_WATER    = C_LIGHT / N_WATER
8  MU_P_THRESHOLD = 0.1185      # GeV/c — muon MRD threshold
9  MAX_PHYSICAL_CM = 512.0      # ANNIE cylinder diagonal (cm)
10
11 # — Particle masses (GeV/c²) —————
12 M_MU      = 0.1056583755
13 M_PI      = 0.13957039
14 ALPHA     = 1 / 137.035999084
15
16 # — Frank-Tamm wavelength grid (300–700 nm) —————
17 LAM_MIN_NM = 300.0
18 LAM_MAX_NM = 700.0
19 N_LAM_BINS = 40
20 LAM_EDGES_NM = np.linspace(LAM_MIN_NM, LAM_MAX_NM, N_LAM_BINS + 1)
21 LAM_CENT_NM  = 0.5 * (LAM_EDGES_NM[:-1] + LAM_EDGES_NM[1:])
22 DLAM_CM      = np.diff(LAM_EDGES_NM) * 1e-7
23 LAM_CENT_CM  = LAM_CENT_NM * 1e-7
24 FT_LIMIT     = 387.0          # Frank-Tamm  $\theta \rightarrow 1$  in water, ph/cm
25
26 # — Colour palette —————
27 C_MU      = '#1f77b4'        # blue — muon overall
28 C_PI      = '#d62728'        # red — pion
29 C_MC      = '#aec7e8'        # lt blue — contained muon
30 C_MU2     = '#ff7f0e'        # orange — punch-through muon
31 C_DECAY   = '#2ca02c'        # green — decay
32 C_INEL    = '#9467bd'        # purple — inelastic
33 C_OTHER   = '#8c564b'        # brown — escapes/other
34
35 # — Geometry helper —————
36 def tof_to_cylinder(x, y, z):
37     """Shortest distance (mm) from point to ANNIE cylinder wall."""
38     r      = np.sqrt(x**2 + y**2)
39     d_barrel = np.clip(R_CYL - r, 0, None)
40     d_endcap = np.clip(np.where(z >= 0, Z_MAX - z, Z_MAX + z), 0, None)
41     return np.minimum(d_barrel, d_endcap)
42
43 # — Relativistic kinematics —————
44 def beta_from_p(p, m):
45     p = np.asarray(p, dtype=float)
46     return p / np.sqrt(p**2 + m**2)
47
48 def gamma_from_p(p, m):
49     p = np.asarray(p, dtype=float)
50     return np.sqrt(1.0 + (p / m)**2)
51
52 def effective_threshold_momentum(m):
53     n_eff = np.average(n_water_dispersion(LAM_CENT_NM), weights=1./LAM_
54     beta_thr = 1.0 / n_eff
55     gamma_thr = 1.0 / np.sqrt(1.0 - beta_thr**2)

```

```

56     return m * beta_thr * gamma_thr, n_eff
57
58 # — Water dispersion (Daimon & Masumura Sellmeier) —————
59 def n_water_dispersion(lam_nm):
60     lam = np.asarray(lam_nm, dtype=float) * 1e-3
61     lam2 = lam**2
62     B = np.array([0.75831, 0.08495, 0.00143])
63     C = np.array([0.01007, 0.08997, 897.0 ])
64     n2 = 1.0 + np.sum(B[:, None] * lam2[None, :],
65                     / (lam2[None, :] - C[:, None])), axis=0)
66     return np.sqrt(n2)
67
68 # — Frank-Tamm instantaneous dN/dx (ph/cm) —————
69 def frank_tamm_per_cm_band(p, m):
70     beta = beta_from_p(p, m)
71     beta2 = np.atleast_1d(beta)**2
72     nlam = n_water_dispersion(LAM_CENT_NM)
73     headroom = np.clip(1.0 - 1.0 / (beta2[:, None] * nlam[None, :])**2), 0
74     d2N = 2.0 * np.pi * ALPHA * headroom / (LAM_CENT_CM[None, :])**2)
75     return np.sum(d2N * DLAM_CM[None, :], axis=1)
76
77 # — Bethe-Bloch dE/dx in water (MeV/cm) —————
78 def bethe_bloch_water_GeV(p_GeV, m_GeV):
79     p_MeV = np.asarray(p_GeV, dtype=float) * 1000.
80     M_MeV = m_GeV * 1000.
81     E_MeV = np.sqrt(p_MeV**2 + M_MeV**2)
82     gamma = E_MeV / M_MeV
83     beta = p_MeV / E_MeV
84     bg = beta * gamma
85     I_water = 79.7e-6
86     Z_over_A = 0.555
87     me = 0.511
88     K = 0.307075
89     x = np.log10(bg)
90     C_s, x0, x1, a, k = -3.5017, 0.2400, 2.8004, 0.09116, 3.4773
91     delta = np.where(x >= x1,
92                     2*np.log(10)*x + C_s,
93                     np.where(x >= x0,
94                             2*np.log(10)*x + C_s + a*(x1 - x)**k,
95                             0.0))
96     Tmax = (2*me*beta**2*gamma**2
97            / (1 + 2*gamma*me/M_MeV + (me/M_MeV)**2))
98     ln_t = np.log(2*me*beta**2*gamma**2*Tmax / I_water**2)
99     return K * Z_over_A / beta**2 * (0.5*ln_t - beta**2 - delta/2)
100
101 # — Track-averaged Frank-Tamm (matches cher_total/track_len) —————
102 def frank_tamm_track_averaged(p0_array, m_GeV, n_steps=200):
103     p0_array = np.asarray(p0_array, dtype=float)
104     result = np.empty_like(p0_array)
105     n_eff = np.average(n_water_dispersion(LAM_CENT_NM), weights=1./LAM
106     beta_thr = 1.0 / n_eff
107     p_thr = m_GeV * beta_thr / np.sqrt(1.0 - beta_thr**2)
108     for i, p0 in enumerate(p0_array):
109         if p0 <= p_thr:
110             result[i] = 0.0
111         continue

```

```

112     E0      = np.sqrt(p0**2 + m_GeV**2)
113     E_min   = max(np.sqrt(p_thr**2 + m_GeV**2) * 0.95, m_GeV * 1.001)
114     E_grid  = np.linspace(E0, E_min, n_steps)
115     total_photons = 0.0
116     total_length = 0.0
117     for j in range(len(E_grid) - 1):
118         E_mid = 0.5 * (E_grid[j] + E_grid[j+1])
119         dE     = abs(E_grid[j] - E_grid[j+1])
120         p_mid  = np.sqrt(max(E_mid**2 - m_GeV**2, 0.))
121         if p_mid <= 0:
122             continue
123         dEdx_mev_cm = bethe_bloch_water_GeV(p_mid, m_GeV)
124         if dEdx_mev_cm <= 0:
125             continue
126         dx_cm = (dE * 1000.) / dEdx_mev_cm
127         dNdx  = frank_tamm_per_cm_band(np.array([p_mid]), m_GeV)[0]
128         total_photons += dNdx * dx_cm
129         total_length  += dx_cm
130     result[i] = total_photons / total_length if total_length > 0 else
131     return result
132
133 # — Binned quantiles —
134 def binned_quantiles(x, y, bins, qs=(0.025, 0.16, 0.50, 0.84, 0.975), min
135     x = np.asarray(x); y = np.asarray(y)
136     centers = 0.5 * (bins[:-1] + bins[1:])
137     out     = {q: np.full(len(centers), np.nan) for q in qs}
138     counts  = np.zeros(len(centers), dtype=int)
139     idx     = np.digitize(x, bins) - 1
140     for i in range(len(centers)):
141         m = (idx == i) & np.isfinite(x) & np.isfinite(y)
142         counts[i] = m.sum()
143         if counts[i] >= min_count:
144             for q in qs:
145                 out[q][i] = np.quantile(y[m], q)
146     return centers, out, counts
147
148 # —  $\pi^+$  fate simplifier —
149 def simplify_fate(p):
150     if 'Decay' in p: return 'Decay'
151     if 'Inelastic' in p: return 'Inelastic'
152     return 'Escapes'
153
154 # — Signal filter constants —
155 LATE_OUTLIER_EID = 23630 # single large-TOF outlier event
156
157 # — EventTree columns needed —
158 SIG_COLS = [
159     'event_id',
160     'mu_cher_total', 'mu_track_len', 'mu_time_stop', 'mu_contained',
161     'pi_cher_total', 'pi_track_len', 'pi_time_end', 'pi_contained',
162     'mu_p0', 'pi_p0', 'open_angle_deg', 'Enu',
163     'mu_final_process', 'mu_KE0',
164     'pi_final_process', 'pi_track_id',
165     'n_secondary_pi',
166     'smu_t_start', 'smu_KE0', 'smu_time_stop', 'smu_contained',
167     'smu_cher_total',

```

```
168     'michel_e_present', 'michel_e_time_start', 'michel_e_KE0',  
169     'michel_e_cher_total',  
170 ]  
171  
172 print('Definitions OK')
```

Definitions OK

Cell 4 — Computations (load data, derived columns, signal filter, StepTree prep)

In [10]:

```

1  # — 4a: Load EventTree —————
2  with uproot.open(SHUF_PATH) as f:
3      et_key = sorted([k for k in f.keys() if 'EventTree' in k])[-1]
4      ev_all = f[et_key].arrays(SIG_COLS, library='pd')
5
6  print(f"Loaded {len(ev_all):,} events from {et_key}")
7
8  # — 4b: Type fixes —————
9  ev_all['mu_contained'] = ev_all['mu_contained'].astype(bool)
10 ev_all['pi_contained'] = ev_all['pi_contained'].astype(bool)
11 ev_all['michel_e_present'] = ev_all['michel_e_present'].astype(bool)
12 ev_all['smu_contained'] = ev_all['smu_contained'].astype(bool)
13
14 for col in ['mu_final_process', 'pi_final_process']:
15     ev_all[col] = ev_all[col].astype(str).str.strip().str.rstrip('\x00')
16
17 # — 4c: Unit fix - track_len stored in mm, convert to cm —————
18 if ev_all['mu_track_len'].max() > MAX_PHYSICAL_CM * 2:
19     ev_all['mu_track_len'] = ev_all['mu_track_len'] / 10.0
20     ev_all['pi_track_len'] = ev_all['pi_track_len'] / 10.0
21     print(" track_len branches were in mm - converted to cm")
22
23 # — 4d: Derived columns (all events) —————
24 ev_all['fate'] = ev_all['pi_final_process'].apply(simplify_fate)
25 ev_all['mu_cher_per_cm'] = np.where(ev_all['mu_track_len'] > 0,
26                                     ev_all['mu_cher_total'] / ev_all['mu_track_len'],
27                                     0)
28 ev_all['pi_cher_per_cm'] = np.where(ev_all['pi_track_len'] > 0,
29                                     ev_all['pi_cher_total'] / ev_all['pi_track_len'],
30                                     0)
31
32 # — 4e: Containment masks —————
33 mu_cont = ev_all[ev_all['mu_contained']]
34 mu_uncont = ev_all[~ev_all['mu_contained']]
35
36 # — 4f: Signal filter (MRD punch-through, above threshold) —————
37 cut_threshold = ev_all['mu_p0'] > MU_P_THRESHOLD
38 cut_punchthru = ~ev_all['mu_contained']
39 cut_signal = cut_threshold & cut_punchthru
40
41 ev_sig = ev_all[cut_signal].copy().set_index('event_id')
42 ev_sig['ratio'] = ev_sig['pi_cher_total'] / ev_sig['mu_cher_total']
43 ev_sig['mu_cher_per_cm'] = ev_sig['mu_cher_total'] / ev_sig['mu_track_len']
44 ev_sig['pi_cher_per_cm'] = ev_sig['pi_cher_total'] / ev_sig['pi_track_len']
45 ev_sig['dt_pi_after_mu'] = ev_sig['pi_time_end'] - ev_sig['mu_time_start']
46 ev_sig['mu_plus_lifetime'] = np.where(
47     ev_sig['smu_t_start'] > 0,
48     ev_sig['smu_time_stop'] - ev_sig['smu_t_start'],
49     np.nan
50 )
51
52 # — 4g: True / fake Michel classification —————
53 ev_sig['true_michel'] = (
54     ev_sig['michel_e_present'] &
55     (ev_sig['fate'] == 'Decay') &
56     (ev_sig['smu_t_start'] > 0) &
57     (ev_sig['n_secondary_pi'] == 0)

```

```

56 )
57 ev_sig['fake_michel'] = (
58     ev_sig['michel_e_present'] &
59     (ev_sig['n_secondary_pi'] > 0)
60 )
61
62 # — 4h: Convenience sets / Lookups used by StepTree cells —————
63 sig_ids      = set(np.int32(list(ev_sig.index)))
64 decay_ids    = set(np.int32(
65     ev_sig[ev_sig['true_michel'] | (ev_sig['fate']=='Decay')].index.tolist()
66 pi_tid_lookup = ev_sig['pi_track_id'].to_dict()
67
68 # muon exit time for shoulder plot
69 ev_sig['mu_water_exit_ns'] = ev_sig['mu_time_stop']
70 mu_exit_lookup = ev_sig[['mu_water_exit_ns']]
71
72 # outlier-removed copy
73 ev_sig_no23630 = ev_sig[ev_sig.index != LATE_OUTLIER_EID].copy()
74
75 # — Summary —————
76 n_total      = len(ev_all)
77 n_subthresh  = (~cut_threshold).sum()
78 n_contained  = cut_threshold.sum() - cut_signal.sum()
79 n_signal     = len(ev_sig)
80
81 print('='*60)
82 print(' SIGNAL FILTER SUMMARY')
83 print('='*60)
84 print(f" Total events           : {n_total:,}")
85 print(f" Sub-threshold muon      : {n_subthresh:,} ({n_subthresh/n_total*100:.1f}%)")
86 print(f" Contained muon (stops)  : {n_contained:,} ({n_contained/n_total*100:.1f}%)")
87 print(f" Signal events           : {n_signal:,} ({n_signal/n_total*100:.1f}%)")
88 print()
89 print(f"  $\mu^-$  containment fraction : {ev_all['mu_contained'].mean()*100:.1f} %")
90 print(f"  $\pi^+$  containment fraction  : {ev_all['pi_contained'].mean()*100:.1f} %")
91 print(f" True Michel events (sig) : {ev_sig['true_michel'].sum():,}")
92 print(f" Outlier event {LATE_OUTLIER_EID} excluded in ev_sig_no23630")

```

Loaded 30,000 events from EventTree;1

track_len branches were in mm – converted to cm

```

=====
SIGNAL FILTER SUMMARY
=====

```

```

Total events           : 30,000
Sub-threshold muon     : 853 (2.8%)
Contained muon (stops) : 17,118 (57.1%)
Signal events          : 12,029 (40.1%)

```

```

 $\mu^-$  containment fraction : 59.9%
 $\pi^+$  containment fraction : 99.9%
True Michel events (sig) : 6,265
Outlier event 23630 excluded in ev_sig_no23630

```


Plot 1 — Prompt light yield + timing (2×2 panel)

In [11]:

```

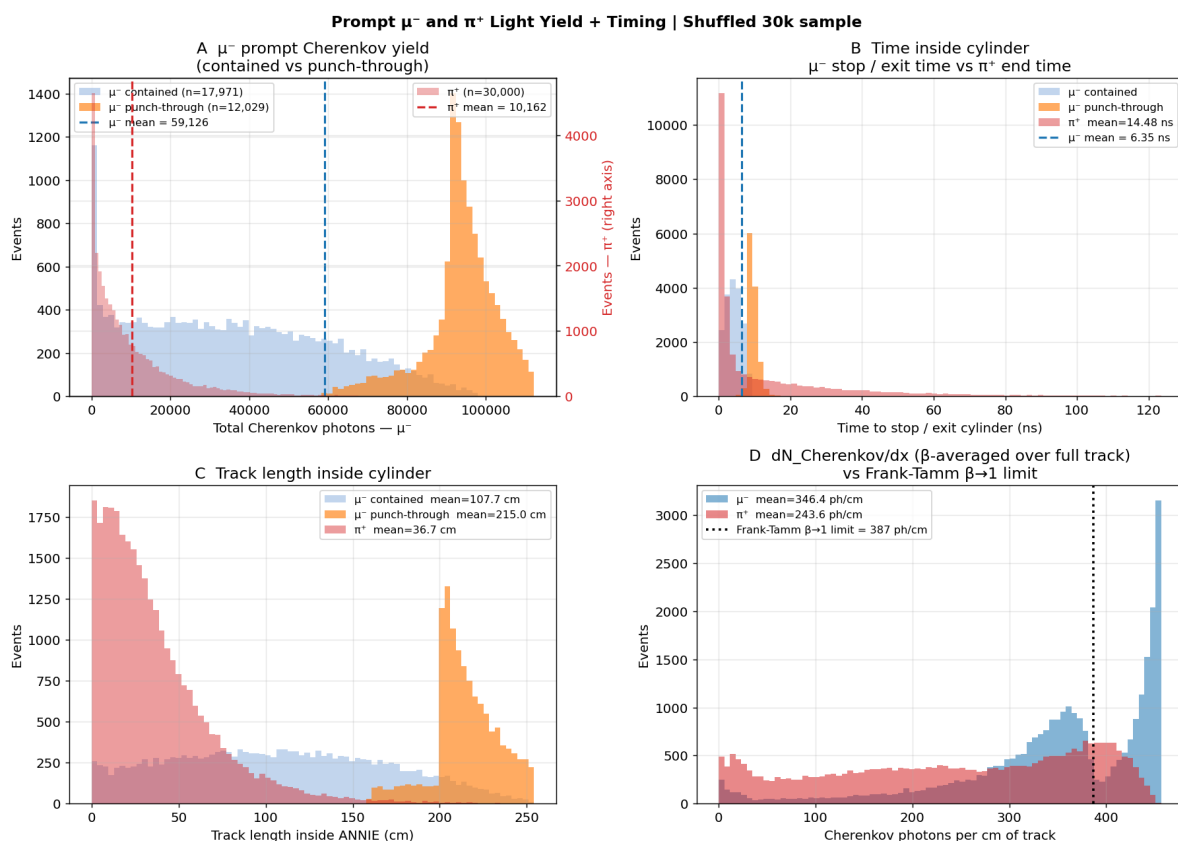
1 fig, axes = plt.subplots(2, 2, figsize=(14, 10))
2 fig.suptitle('Prompt  $\mu^-$  and  $\pi^+$  Light Yield + Timing | Shuffled 30k sample
3               fontsize=13, fontweight='bold')
4
5 # — Panel A: Total Cherenkov yield —————
6 ax = axes[0, 0]
7 bins_y = np.linspace(0, ev_all['mu_cher_total'].quantile(0.995), 80)
8 ax.hist(mu_cont['mu_cher_total'], bins=bins_y, color=C_MC, alpha=0.75,
9         label=f' $\mu^-$  contained (n={len(mu_cont):,})')
10 ax.hist(mu_uncont['mu_cher_total'], bins=bins_y, color=C_MU2, alpha=0.65,
11         label=f' $\mu^-$  punch-through (n={len(mu_uncont):,})')
12 ax.axvline(ev_all['mu_cher_total'].mean(), color=C_MU, lw=1.8, ls='--',
13            label=f' $\mu^-$  mean = {ev_all["mu_cher_total"].mean():,.0f}')
14 ax.set_xlabel('Total Cherenkov photons -  $\mu^-$ ')
15 ax.set_ylabel('Events')
16 ax.set_title('A  $\mu^-$  prompt Cherenkov yield\n(contained vs punch-through)')
17 ax.legend(fontsize=9)
18 ax2 = ax.twinx()
19 bins_pi = np.linspace(0, ev_all['pi_cher_total'].quantile(0.995), 80)
20 ax2.hist(ev_all['pi_cher_total'], bins=bins_pi, color=C_PI, alpha=0.35,
21         label=f' $\pi^+$  (n={len(ev_all):,})')
22 ax2.axvline(ev_all['pi_cher_total'].mean(), color=C_PI, lw=1.8, ls='--',
23            label=f' $\pi^+$  mean = {ev_all["pi_cher_total"].mean():,.0f}')
24 ax2.set_ylabel('Events -  $\pi^+$  (right axis)', color=C_PI)
25 ax2.tick_params(axis='y', labelcolor=C_PI)
26 ax2.legend(fontsize=9, loc='upper right')
27
28 # — Panel B: Time to stop / exit —————
29 ax = axes[0, 1]
30 t_max = max(ev_all['mu_time_stop'].quantile(0.995),
31            ev_all['pi_time_end'].quantile(0.995))
32 bins_t = np.linspace(0, t_max, 80)
33 ax.hist(mu_cont['mu_time_stop'], bins=bins_t, color=C_MC, alpha=0.75,
34         label=f' $\mu^-$  contained mean={mu_cont["mu_time_stop"].mean():.1f} ns')
35 ax.hist(mu_uncont['mu_time_stop'], bins=bins_t, color=C_MU2, alpha=0.65,
36         label=f' $\mu^-$  punch-through mean={mu_uncont["mu_time_stop"].mean():.1f} ns')
37 ax.hist(ev_all['pi_time_end'], bins=bins_t, color=C_PI, alpha=0.45,
38         label=f' $\pi^+$  mean={ev_all["pi_time_end"].mean():.2f} ns')
39 ax.axvline(ev_all['mu_time_stop'].mean(), color=C_MU, lw=1.8, ls='--',
40            label=f' $\mu^-$  mean = {ev_all["mu_time_stop"].mean():.2f} ns')
41 ax.set_xlabel('Time to stop / exit cylinder (ns)')
42 ax.set_ylabel('Events')
43 ax.set_title('B Time inside cylinder\n $\mu^-$  stop / exit time vs  $\pi^+$  end time')
44 ax.legend(fontsize=9)
45
46 # — Panel C: Track Length —————
47 ax = axes[1, 0]
48 len_max = max(ev_all['mu_track_len'].quantile(0.995),
49              ev_all['pi_track_len'].quantile(0.995))
50 bins_l = np.linspace(0, len_max, 80)
51 ax.hist(mu_cont['mu_track_len'], bins=bins_l, color=C_MC, alpha=0.75,
52         label=f' $\mu^-$  contained mean={mu_cont["mu_track_len"].mean():.1f} cm')
53 ax.hist(mu_uncont['mu_track_len'], bins=bins_l, color=C_MU2, alpha=0.65,
54         label=f' $\mu^-$  punch-through mean={mu_uncont["mu_track_len"].mean():.1f} cm')
55 ax.hist(ev_all['pi_track_len'], bins=bins_l, color=C_PI, alpha=0.45,
56         label=f' $\pi^+$  mean={ev_all["pi_track_len"].mean():.1f} cm')
57 ax.set_xlabel('Track length inside ANNIE (cm)')

```

```

56 ax.set_ylabel('Events')
57 ax.set_title('C Track length inside cylinder')
58 ax.legend(fontsize=9)
59
60 # — Panel D: dN_Cherenkov/dx vs Frank-Tamm Limit —————
61 ax = axes[1, 1]
62 valid_mu = ev_all['mu_cher_per_cm'].dropna()
63 valid_pi = ev_all['pi_cher_per_cm'].dropna()
64 dndx_max = max(valid_mu.quantile(0.995), valid_pi.quantile(0.995), FT_LIMIT)
65 bins_d = np.linspace(0, dndx_max, 80)
66 ax.hist(valid_mu, bins=bins_d, color=C_MU, alpha=0.55,
67         label=f' $\mu^-$  mean={valid_mu.mean():.1f} ph/cm')
68 ax.hist(valid_pi, bins=bins_d, color=C_PI, alpha=0.55,
69         label=f' $\pi^+$  mean={valid_pi.mean():.1f} ph/cm')
70 ax.axvline(FT_LIMIT, color='black', lw=2, ls=':',
71           label=f'Frank-Tamm  $\beta \rightarrow 1$  limit = {FT_LIMIT:.0f} ph/cm')
72 ax.set_xlabel('Cherenkov photons per cm of track')
73 ax.set_ylabel('Events')
74 ax.set_title('D dN_Cherenkov/dx ( $\beta$ -averaged over full track)\nvs Frank-Ta
75 ax.legend(fontsize=9)
76
77 plt.tight_layout()
78 # fig.savefig(PNG_DIR / 'plot1_prompt_yield_timing.png', dpi=600, bbox_inches=
79 plt.show()

```



Plot 2 — π^+/μ^- Cherenkov ratio vs opening angle (1×3 panel)

In [12]:

```

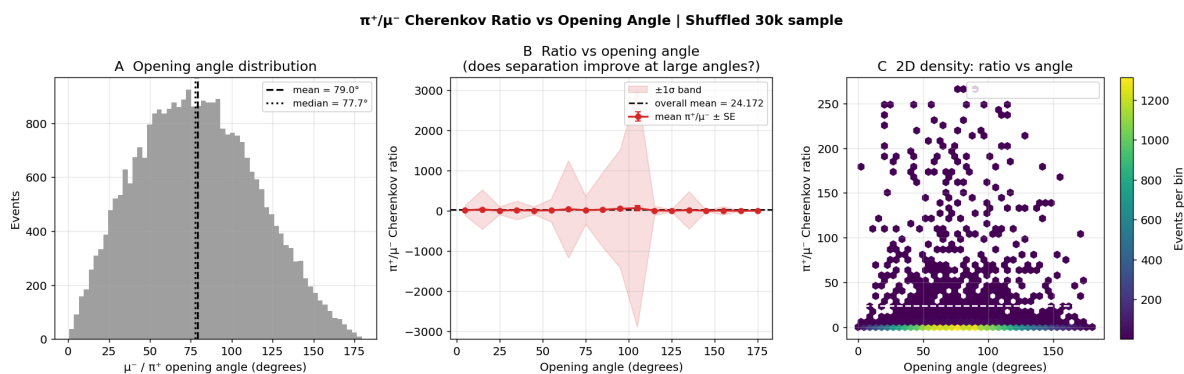
1  # Re-load just the columns needed
2  angle_cols = ['open_angle_deg', 'Enu', 'mu_p0', 'pi_p0',
3               'mu_cher_total', 'pi_cher_total', 'mu_contained', 'pi_contained']
4  with uproot.open(SHUF_PATH) as f:
5      et_key = sorted([k for k in f.keys() if 'EventTree' in k])[-1]
6      ev2 = f[et_key].arrays(angle_cols, library='pd')
7  ev2['mu_contained'] = ev2['mu_contained'].astype(bool)
8  ev2['pi_contained'] = ev2['pi_contained'].astype(bool)
9  ev2['ratio'] = np.where(ev2['mu_cher_total'] > 0,
10                        ev2['pi_cher_total'] / ev2['mu_cher_total'],
11                        0)
12  angle_bins = np.arange(0, 181, 10)
13  angle_cents = (angle_bins[:-1] + angle_bins[1:]) / 2
14  ev2['angle_bin'] = pd.cut(ev2['open_angle_deg'], bins=angle_bins, labels=angle_cents)
15  bin_stats = (ev2.groupby('angle_bin', observed=True)['ratio']
16              .agg(mean='mean', median='median', std='std', count='count')
17              .reset_index())
18  bin_stats['se'] = bin_stats['std'] / np.sqrt(bin_stats['count'])
19
20  fig, axes = plt.subplots(1, 3, figsize=(16, 5))
21  fig.suptitle('π+/μ- Cherenkov Ratio vs Opening Angle | Shuffled 30k sample',
22              fontsize=13, fontweight='bold')
23
24  # — Panel A: Opening angle distribution —————
25  ax = axes[0]
26  ax.hist(ev2['open_angle_deg'], bins=60, color='#7f7f7f', alpha=0.75)
27  ax.axvline(ev2['open_angle_deg'].mean(), color='black', lw=1.8, ls='--',
28             label=f'mean = {ev2["open_angle_deg"].mean():.1f}°')
29  ax.axvline(ev2['open_angle_deg'].median(), color='black', lw=1.8, ls=':',
30             label=f'median = {ev2["open_angle_deg"].median():.1f}°')
31  ax.set_xlabel('μ- / π+ opening angle (degrees)')
32  ax.set_ylabel('Events')
33  ax.set_title('A Opening angle distribution')
34  ax.legend(fontsize=9)
35
36  # — Panel B: Mean ratio per angle bin —————
37  ax = axes[1]
38  x = bin_stats['angle_bin'].astype(float)
39  ax.errorbar(x, bin_stats['mean'], yerr=bin_stats['se'],
40             fmt='o-', color=C_PI, lw=1.8, ms=5, capsize=3, label='mean π+/μ-')
41  ax.fill_between(x, bin_stats['mean'] - bin_stats['std'],
42                 bin_stats['mean'] + bin_stats['std'],
43                 alpha=0.15, color=C_PI, label='±1σ band')
44  ax.axhline(ev2['ratio'].mean(), color='black', lw=1.5, ls='--',
45             label=f'overall mean = {ev2["ratio"].mean():.3f}')
46  ax.set_xlabel('Opening angle (degrees)')
47  ax.set_ylabel('π+/μ- Cherenkov ratio')
48  ax.set_title('B Ratio vs opening angle\n(does separation improve at large angles?)')
49  ax.legend(fontsize=9)
50
51  # — Panel C: 2D hexbin —————
52  ax = axes[2]
53  valid = ev2.dropna(subset=['ratio', 'open_angle_deg'])
54  hb = ax.hexbin(valid['open_angle_deg'], valid['ratio'],
55                 gridsize=40, cmap='viridis', mincnt=1,

```

```

56         extent=[0, 180, 0, valid['ratio'].quantile(0.995)])
57 plt.colorbar(hb, ax=ax, label='Events per bin')
58 ax.axhline(ev2['ratio'].mean(), color='white', lw=1.5, ls='--',
59           label=f'mean ratio = {ev2["ratio"].mean():.3f}')
60 ax.set_xlabel('Opening angle (degrees)')
61 ax.set_ylabel('π+/μ- Cherenkov ratio')
62 ax.set_title('C 2D density: ratio vs angle')
63 ax.legend(fontsize=9, labelcolor='white')
64
65 plt.tight_layout()
66 # fig.savefig(PNG_DIR / 'plot2_ratio_vs_angle.png', dpi=600, bbox_inches=
67 plt.show()

```



Plot 3 — π⁺ kinematics, fate, and ratio vs opening angle (3×3, signal sample)

In [13]:

```

1  with uproot.open(SHUF_PATH) as f:
2      et_key = sorted([k for k in f.keys() if 'EventTree' in k])[-1]
3      extra_cols = ['pi_final_process', 'mu_p0', 'pi_p0',
4                    'mu_cher_total', 'pi_cher_total', 'open_angle_deg',
5                    'mu_contained', 'mu_KE0']
6      ev_extra = f[et_key].arrays(extra_cols, library='pd')
7  ev_extra['mu_contained'] = ev_extra['mu_contained'].astype(bool)
8  ev_extra['pi_final_process'] = (ev_extra['pi_final_process']
9                                  .astype(str).str.strip().str.rstrip('\x00')
10 # Apply same signal filter
11 sig_mask = (ev_extra['mu_p0'] > MU_P_THRESHOLD) & (~ev_extra['mu_contained'])
12 ev_extra = ev_extra[sig_mask].copy()
13 ev_extra['fate'] = ev_extra['pi_final_process'].apply(simplify_fate)
14 ev_extra['ratio'] = ev_extra['pi_cher_total'] / ev_extra['mu_cher_total']
15
16 fate_colors = {'Decay': C_DECAY, 'Inelastic': C_INEL, 'Escapes': C_OTHER}
17 angle_bins = np.arange(0, 181, 10)
18 angle_cents = (angle_bins[:-1] + angle_bins[1:]) / 2
19
20 fig, axes = plt.subplots(3, 3, figsize=(16, 13))
21 fig.suptitle('π+ Kinematics, Fate, and Ratio vs Opening Angle | MRD Signal')
22             fontsize=13, fontweight='bold')
23
24 # — Row 0: momenta —————
25 ax = axes[0, 0]
26 ax.hist(ev_extra['pi_p0'] * 1000, bins=60, color=C_PI, alpha=0.75)
27 ax.axvline(156.5, color='black', lw=1.8, ls='--', label='π+ threshold 156')
28 ax.axvline(ev_extra['pi_p0'].mean() * 1000, color=C_PI, lw=1.8, ls=':',
29             label=f'mean = {ev_extra["pi_p0"].mean()*1000:.1f} MeV/c')
30 ax.set_xlabel('π+ initial momentum (MeV/c)'); ax.set_ylabel('Events')
31 ax.set_title('A π+ momentum distribution (signal sample)'); ax.legend(fontsize=9)
32
33 ax = axes[0, 1]
34 hb = ax.hexbin(ev_extra['open_angle_deg'], ev_extra['pi_p0']*1000,
35                gridsize=40, cmap='viridis', mincnt=1)
36 plt.colorbar(hb, ax=ax, label='Events per bin')
37 ax.axhline(156.5, color='white', lw=1.8, ls='--', label='π+ threshold')
38 pi_p0_bins = (ev_extra.groupby(
39     pd.cut(ev_extra['open_angle_deg'], bins=angle_bins, labels=angle_cents)
40 )['pi_p0'].median() * 1000)
41 ax.plot(pi_p0_bins.index.astype(float), pi_p0_bins.values,
42         'o-', color='white', lw=1.8, ms=4, label='median π+ p0')
43 ax.set_xlabel('Opening angle (degrees)'); ax.set_ylabel('π+ initial momentum')
44 ax.set_title('B π+ momentum vs opening angle'); ax.legend(fontsize=9)
45
46 ax = axes[0, 2]
47 hb = ax.hexbin(ev_extra['open_angle_deg'], ev_extra['mu_p0']*1000,
48                gridsize=40, cmap='plasma', mincnt=1)
49 plt.colorbar(hb, ax=ax, label='Events per bin')
50 mu_p0_bins = (ev_extra.groupby(
51     pd.cut(ev_extra['open_angle_deg'], bins=angle_bins, labels=angle_cents)
52 )['mu_p0'].median() * 1000)
53 ax.plot(mu_p0_bins.index.astype(float), mu_p0_bins.values,
54         'o-', color='white', lw=1.8, ms=4, label='median μ- p0')
55 ax.set_xlabel('Opening angle (degrees)'); ax.set_ylabel('μ- initial momentum')

```

```

56 ax.set_title('C  $\mu^-$  momentum vs opening angle'); ax.legend(fontsize=9)
57
58 # — Row 1: fate —————
59 ax = axes[1, 0]
60 for fate, grp in ev_extra.groupby('fate'):
61     ax.hist(grp['open_angle_deg'], bins=angle_bins, alpha=0.6,
62             color=fate_colors.get(fate, 'grey'), label=f'{fate} (n={len(g
63 ax.set_xlabel('Opening angle (degrees)'); ax.set_ylabel('Events')
64 ax.set_title('D  $\pi^+$  fate vs opening angle (counts)'); ax.legend(fontsize=
65
66 ax = axes[1, 1]
67 decay_frac, inel_frac, other_frac = [], [], []
68 for lo, hi in zip(angle_bins[:-1], angle_bins[1:]):
69     sl = ev_extra[(ev_extra['open_angle_deg'] >= lo) & (ev_extra['open_an
70     n = len(sl)
71     if n > 0:
72         decay_frac.append((sl['fate']=='Decay').sum() / n)
73         inel_frac.append((sl['fate']=='Inelastic').sum() / n)
74         other_frac.append((sl['fate']=='Escapes').sum() / n)
75     else:
76         decay_frac.append(0); inel_frac.append(0); other_frac.append(0)
77 ax.bar(angle_cents, decay_frac, width=9, color=C_DECAY, alpha=0.8, label=
78 ax.bar(angle_cents, inel_frac, width=9, color=C_INEL, alpha=0.8, bottom
79 ax.bar(angle_cents, other_frac, width=9, color=C_OTHER, alpha=0.8,
80         bottom=np.array(decay_frac)+np.array(inel_frac), label='Escapes')
81 ax.set_xlabel('Opening angle (degrees)'); ax.set_ylabel('Fraction')
82 ax.set_title('E  $\pi^+$  fate fraction vs opening angle'); ax.legend(fontsize=
83
84 ax = axes[1, 2]
85 for fate, grp in ev_extra.groupby('fate'):
86     ratio_bins = grp.groupby(
87         pd.cut(grp['open_angle_deg'], bins=angle_bins, labels=angle_cents
88     )['ratio'].median()
89     ax.plot(ratio_bins.index.astype(float), ratio_bins.values,
90             'o-', color=fate_colors.get(fate, 'grey'), lw=1.8, ms=4, labe
91 ax.set_xlabel('Opening angle (degrees)'); ax.set_ylabel('Median  $\pi^+/\mu^-$  rat
92 ax.set_title('F Median ratio vs angle by  $\pi^+$  fate'); ax.legend(fontsize=9
93
94 # — Row 2: ratio vs  $\mu^-$  momentum —————
95 ax = axes[2, 0]
96 mu_p_bins_edges = np.percentile(ev_extra['mu_p0']*1000, np.linspace(0,100
97 ev_extra['mu_p_bin'] = pd.cut(ev_extra['mu_p0']*1000, bins=mu_p_bins_edge
98 ratio_vs_mup = (ev_extra.groupby('mu_p_bin', observed=True)['ratio']
99                 .agg(['median', 'mean', 'std']).reset_index())
100 cents = [(iv.left+iv.right)/2 for iv in ratio_vs_mup['mu_p_bin']]
101 ax.plot(cents, ratio_vs_mup['median'], 'o-', color=C_MU, lw=1.8, ms=5, l
102 ax.plot(cents, ratio_vs_mup['mean'], 's--', color=C_MU2, lw=1.5, ms=4,
103 ax.set_xlabel('  $\mu^-$  initial momentum (MeV/c)'); ax.set_ylabel('  $\pi^+/\mu^-$  Cheren
104 ax.set_title('G Ratio vs  $\mu^-$  momentum'); ax.legend(fontsize=9)
105
106 ax = axes[2, 1]
107 hb = ax.hexbin(ev_extra['mu_p0']*1000,
108                ev_extra['ratio'].clip(upper=ev_extra['ratio'].quantile(0.
109                gridsize=40, cmap='viridis', mincnt=1)
110 plt.colorbar(hb, ax=ax, label='Events per bin')
111 ax.set_xlabel('  $\mu^-$  initial momentum (MeV/c)'); ax.set_ylabel('  $\pi^+/\mu^-$  Cheren

```



```
112 ax.set_title('H 2D density: ratio vs  $\mu^-$  momentum')
113
114 ax = axes[2, 2]
115 ax.hist2d(ev_extra['open_angle_deg'], ev_extra['pi_p0']*1000,
116           bins=[angle_bins, 60], cmap='viridis')
117 ax.set_xlabel('Opening angle (degrees)'); ax.set_ylabel('π+ initial momen
118 ax.set_title('I 2D: opening angle vs π+ momentum')
119
120 plt.tight_layout()
121 # fig.savefig(PNG_DIR / 'plot3_kinematics_fate_angle.png', dpi=600, bbox_
122 plt.show()
```

C:\Users\Dylon Fleming\AppData\Local\Temp\ipykernel_9352\2141068125.py:38: FutureWarning: Index.ravel returning ndarray is deprecated; in a future version this will return a view on self.

```
pi_p0_bins = (ev_extra.groupby(
```

C:\Users\Dylon Fleming\AppData\Local\Temp\ipykernel_9352\2141068125.py:50: FutureWarning: Index.ravel returning ndarray is deprecated; in a future version this will return a view on self.

```
mu_p0_bins = (ev_extra.groupby(
```

C:\Users\Dylon Fleming\AppData\Local\Temp\ipykernel_9352\2141068125.py:86: FutureWarning: Index.ravel returning ndarray is deprecated; in a future version this will return a view on self.

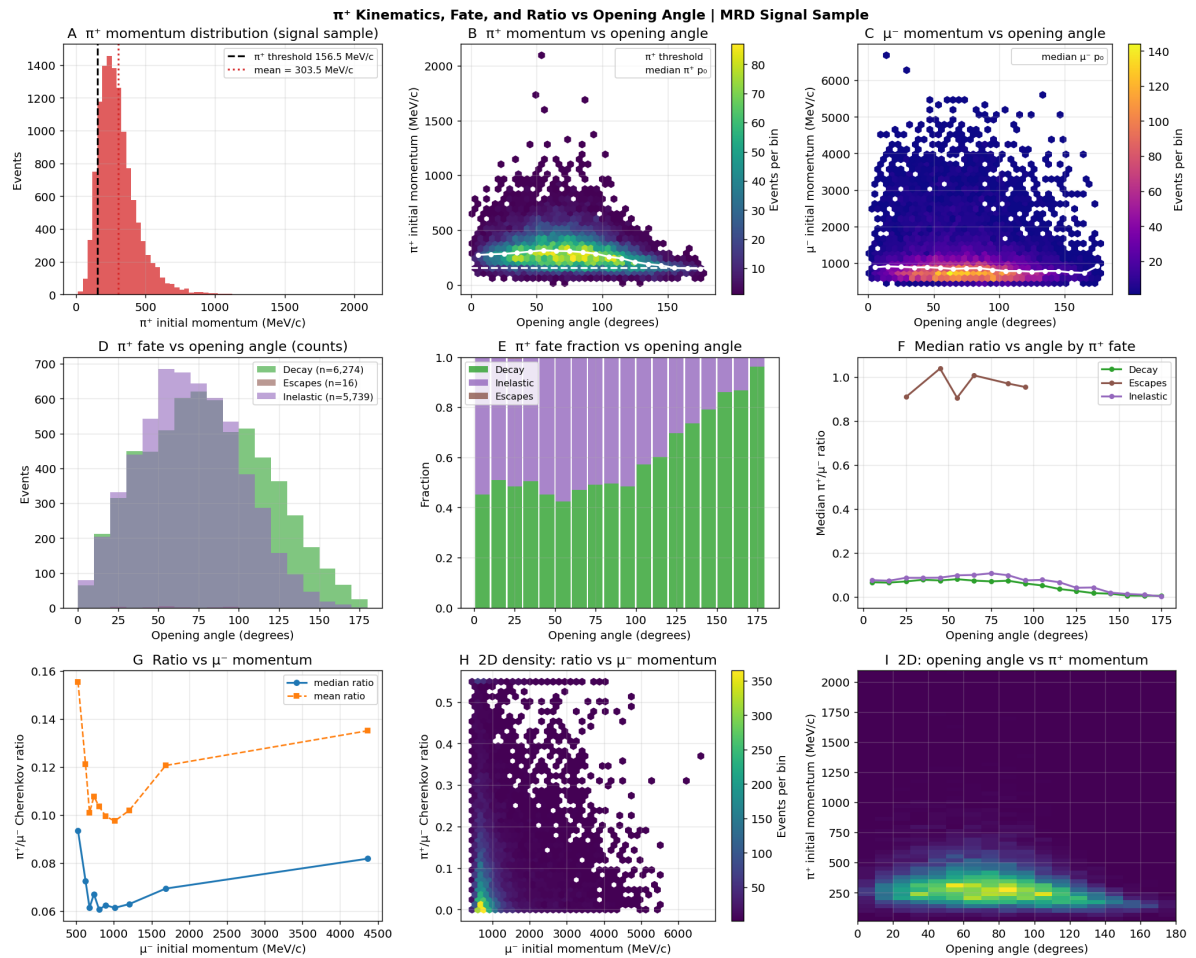
```
ratio_bins = grp.groupby(
```

C:\Users\Dylon Fleming\AppData\Local\Temp\ipykernel_9352\2141068125.py:86: FutureWarning: Index.ravel returning ndarray is deprecated; in a future version this will return a view on self.

```
ratio_bins = grp.groupby(
```

C:\Users\Dylon Fleming\AppData\Local\Temp\ipykernel_9352\2141068125.py:86: FutureWarning: Index.ravel returning ndarray is deprecated; in a future version this will return a view on self.

```
ratio_bins = grp.groupby(
```



Plot 4 — μ^- / π^+ momentum, β , γ comparison (1x4 panel)

In [14]:

```

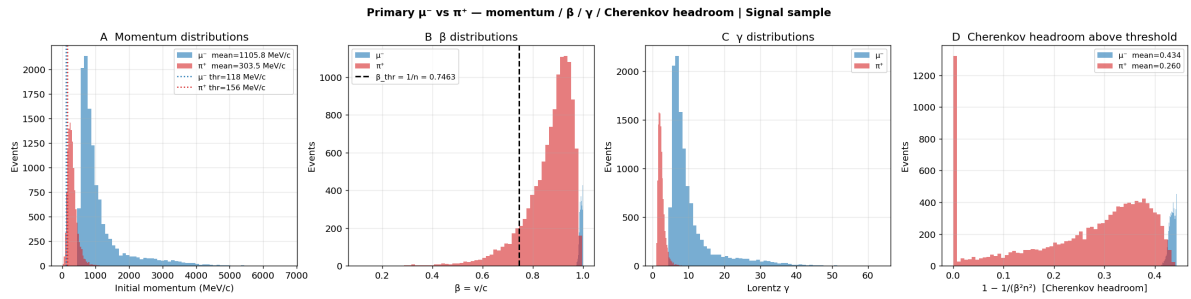
1 BETA_THR = 1.0 / N_WATER
2 GAMMA_THR = 1.0 / np.sqrt(1.0 - BETA_THR**2)
3 PTHR_MU = M_MU * BETA_THR * GAMMA_THR
4 PTHR_PI = M_PI * BETA_THR * GAMMA_THR
5
6 ev_sig['mu_beta'] = beta_from_p(ev_sig['mu_p0'], M_MU)
7 ev_sig['pi_beta'] = beta_from_p(ev_sig['pi_p0'], M_PI)
8 ev_sig['mu_gamma'] = gamma_from_p(ev_sig['mu_p0'], M_MU)
9 ev_sig['pi_gamma'] = gamma_from_p(ev_sig['pi_p0'], M_PI)
10
11 def cherenkov_proxy(beta, n=N_WATER):
12     val = 1.0 - 1.0 / (beta**2 * n**2)
13     return np.where(beta * n > 1, val, 0.0)
14
15 ev_sig['mu_cher_proxy'] = cherenkov_proxy(ev_sig['mu_beta'])
16 ev_sig['pi_cher_proxy'] = cherenkov_proxy(ev_sig['pi_beta'])
17
18 fig, axes = plt.subplots(1, 4, figsize=(20, 5))
19 fig.suptitle('Primary  $\mu^-$  vs  $\pi^+$  - momentum /  $\beta$  /  $\gamma$  / Cherenkov headroom | S
20             fontsize=13, fontweight='bold')
21
22 # Panel A: momentum
23 ax = axes[0]
24 ax.hist(ev_sig['mu_p0']*1000, bins=60, color=C_MU, alpha=0.6,
25         label=f' $\mu^-$  mean={ev_sig["mu_p0"].mean()*1000:.1f} MeV/c')
26 ax.hist(ev_sig['pi_p0']*1000, bins=60, color=C_PI, alpha=0.6,
27         label=f' $\pi^+$  mean={ev_sig["pi_p0"].mean()*1000:.1f} MeV/c')
28 ax.axvline(PTHR_MU*1000, color=C_MU, lw=1.5, ls=':', label=f' $\mu^-$  thr={PTHR
29 ax.axvline(PTHR_PI*1000, color=C_PI, lw=1.5, ls=':', label=f' $\pi^+$  thr={PTHR
30 ax.set_xlabel('Initial momentum (MeV/c)'); ax.set_ylabel('Events')
31 ax.set_title('A Momentum distributions'); ax.legend(fontsize=9)
32
33 # Panel B: beta
34 ax = axes[1]
35 ax.hist(ev_sig['mu_beta'], bins=60, color=C_MU, alpha=0.6, label=' $\mu^-$ ')
36 ax.hist(ev_sig['pi_beta'], bins=60, color=C_PI, alpha=0.6, label=' $\pi^+$ ')
37 ax.axvline(BETA_THR, color='black', lw=1.8, ls='--', label=f' $\beta_{thr} = 1/n$ 
38 ax.set_xlabel(' $\beta = v/c$ '); ax.set_ylabel('Events')
39 ax.set_title('B  $\beta$  distributions'); ax.legend(fontsize=9)
40
41 # Panel C: gamma
42 ax = axes[2]
43 ax.hist(ev_sig['mu_gamma'], bins=60, color=C_MU, alpha=0.6, label=' $\mu^-$ ')
44 ax.hist(ev_sig['pi_gamma'], bins=60, color=C_PI, alpha=0.6, label=' $\pi^+$ ')
45 ax.set_xlabel('Lorentz  $\gamma$ '); ax.set_ylabel('Events')
46 ax.set_title('C  $\gamma$  distributions'); ax.legend(fontsize=9)
47
48 # Panel D: Cherenkov headroom  $1 - 1/(\beta^2 n^2)$ 
49 ax = axes[3]
50 ax.hist(ev_sig['mu_cher_proxy'], bins=60, color=C_MU, alpha=0.6,
51         label=f' $\mu^-$  mean={ev_sig["mu_cher_proxy"].mean():.3f}')
52 ax.hist(ev_sig['pi_cher_proxy'], bins=60, color=C_PI, alpha=0.6,
53         label=f' $\pi^+$  mean={ev_sig["pi_cher_proxy"].mean():.3f}')
54 ax.set_xlabel('1 - 1/(\beta^2 n^2) [Cherenkov headroom]'); ax.set_ylabel('Event
55 ax.set_title('D Cherenkov headroom above threshold'); ax.legend(fontsize=

```

```

56
57 plt.tight_layout()
58 # fig.savefig(PNG_DIR / 'plot4_beta_gamma_headroom.png', dpi=600, bbox_inches=
59 plt.show()

```



Plot 5 — Analytic Cherenkov model validation (Frank-Tamm vs Geant4, 2×2)

In [15]:

```

1  # Load needed columns from EventTree
2  ft_cols = ['event_id', 'mu_p0', 'pi_p0',
3             'mu_cher_total', 'pi_cher_total',
4             'mu_track_len', 'pi_track_len',
5             'mu_final_process', 'pi_final_process']
6  with uproot.open(SHUF_PATH) as f:
7      tree = choose_best_tree(f, 'EventTree')
8      ev_ft = tree.arrays([c for c in ft_cols if c in tree.keys()], library=
9
10     for c in ['mu_p0', 'pi_p0', 'mu_cher_total', 'pi_cher_total', 'mu_track_len',
11              ev_ft[c] = pd.to_numeric(ev_ft[c], errors='coerce')
12     for c in ['mu_final_process', 'pi_final_process']:
13         ev_ft[c] = ev_ft[c].astype(str).str.strip().str.rstrip('\x00')
14     if ev_ft['mu_track_len'].max() > 600:
15         ev_ft['mu_track_len'] = ev_ft['mu_track_len'] / 10.0
16         ev_ft['pi_track_len'] = ev_ft['pi_track_len'] / 10.0
17     ev_ft['mu_cher_per_cm'] = np.where(ev_ft['mu_track_len'] > 0, ev_ft['mu_cher
18     ev_ft['pi_cher_per_cm'] = np.where(ev_ft['pi_track_len'] > 0, ev_ft['pi_cher
19
20     mu_mask = np.isfinite(ev_ft['mu_p0']) & np.isfinite(ev_ft['mu_cher_pe
21     pi_mask = np.isfinite(ev_ft['pi_p0']) & np.isfinite(ev_ft['pi_cher_pe
22     pi_decay_mask = pi_mask & (ev_ft['pi_final_process'] == 'Decay')
23     pi_inel_mask = pi_mask & ev_ft['pi_final_process'].str.contains('Inelast
24
25     pmax = max(ev_ft['mu_p0'].quantile(0.995), ev_ft['pi_p0'].quantile(0.9
26     p_grid = np.linspace(0.001, pmax * 1.05, 300)
27     print('Computing track-averaged Frank-Tamm curves (~10 s)...')
28     mu_ft = frank_tamm_track_averaged(p_grid, M_MU)
29     pi_ft = frank_tamm_track_averaged(p_grid, M_PI)
30     PTHR_MU_EFF, _ = effective_threshold_momentum(M_MU)
31     PTHR_PI_EFF, _ = effective_threshold_momentum(M_PI)
32
33     p_bins = np.linspace(0., pmax*1.05, 40)
34     mu_pc, mu_q, _ = binned_quantiles(ev_ft.loc[mu_mask, 'mu_p0'], ev_ft.loc[mu
35     pi_pc, pi_q, _ = binned_quantiles(ev_ft.loc[pi_mask, 'pi_p0'], ev_ft.loc[pi
36     pi_d_pc, pi_d_q, _ = binned_quantiles(ev_ft.loc[pi_decay_mask, 'pi_p0'], ev
37     pi_i_pc, pi_i_q, _ = binned_quantiles(ev_ft.loc[pi_inel_mask, 'pi_p0'], ev
38
39     fig, axes = plt.subplots(2, 2, figsize=(14, 10))
40     fig.suptitle('Analytic Frank-Tamm vs Geant4 dN_Chер/dx | Full 30k sample'
41                 fontsize=13, fontweight='bold')
42
43     def plot_panel(ax, pc, q, ft_p, ft_y, label, color, title):
44         ax.fill_between(pc, q[0.025], q[0.975], alpha=0.15, color=color)
45         ax.fill_between(pc, q[0.16], q[0.84], alpha=0.25, color=color)
46         ax.plot(pc, q[0.50], 'o-', color=color, lw=1.8, ms=4, label=f'{label}')
47         ax.plot(ft_p, ft_y, '--', color='black', lw=2, label='Frank-Tamm (trac
48         ax.set_xlabel('Initial momentum (GeV/c)'); ax.set_ylabel('dN/dx (ph/cm
49         ax.set_title(title); ax.legend(fontsize=9)
50
51     plot_panel(axes[0,0], mu_pc, mu_q, p_grid, mu_ft, 'μ-', C_MU, 'A μ- dN/dx
52     plot_panel(axes[0,1], pi_pc, pi_q, p_grid, pi_ft, 'π+ all', C_PI, 'B π+ c
53     plot_panel(axes[1,0], pi_d_pc, pi_d_q, p_grid, pi_ft, 'π+ decay', C_DECAY
54         'C π+ decay only – matches Frank-Tamm')
55     plot_panel(axes[1,1], pi_i_pc, pi_i_q, p_grid, pi_ft, 'π+ inelastic', C_I

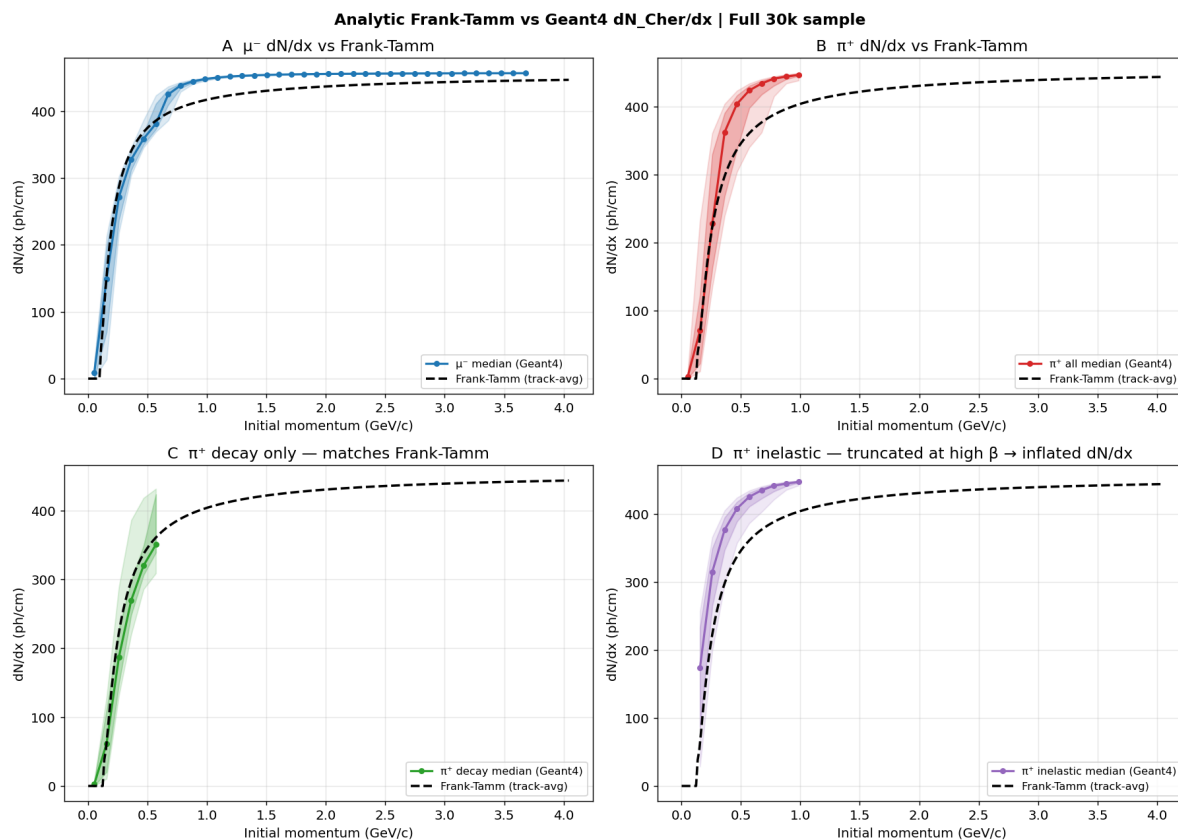
```

```

56         'D  $\pi^+$  inelastic — truncated at high  $\beta \rightarrow$  inflated  $dN/dx$ ')
57
58 plt.tight_layout()
59 # fig.savefig(PNG_DIR / 'plot5_frank_tamm_validation.png', dpi=600, bbox_
60 plt.show()

```

Computing track-averaged Frank-Tamm curves (~ 10 s)...

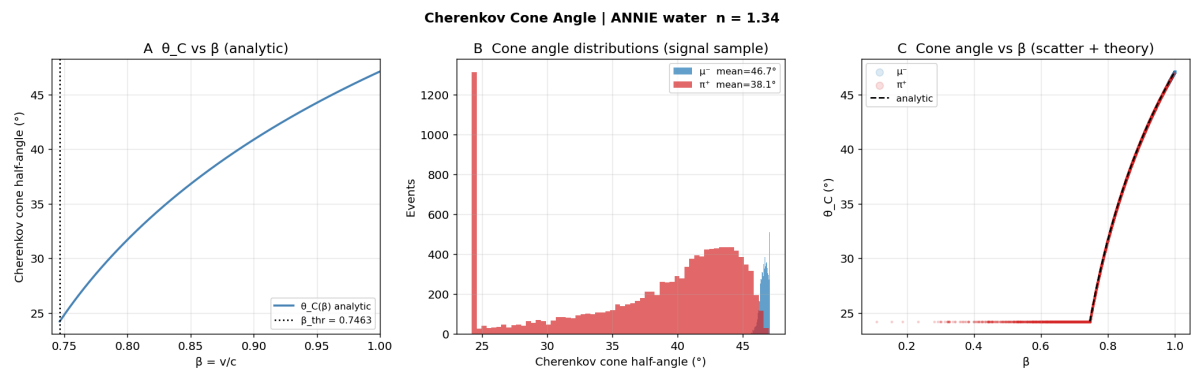


Plot 6 — Cherenkov cone angle vs β (Frank-Tamm, analytic)

```

In [16]: 1 beta_grid = np.linspace(BETA_THR + 1e-5, 0.9999, 400)
2 n_eff = np.average(n_water_dispersion(LAM_CENT_NM), weights=1./LAM_CEN
3 cos_theta = 1.0 / (beta_grid * n_eff)
4 cos_theta = np.clip(cos_theta, -1, 1)
5 theta_deg = np.degrees(np.arccos(cos_theta))
6
7 mu_beta_sig = beta_from_p(ev_sig['mu_p0'].values, M_MU)
8 pi_beta_sig = beta_from_p(ev_sig['pi_p0'].values, M_PI)
9
10 def cone_angle(beta):
11     cos_t = np.clip(1.0/(np.asarray(beta)*n_eff), -1, 1)
12     return np.degrees(np.arccos(cos_t))
13
14 mu_cone = cone_angle(np.clip(mu_beta_sig, BETA_THR+1e-8, 0.9999))
15 pi_cone = cone_angle(np.clip(pi_beta_sig, BETA_THR+1e-8, 0.9999))
16
17 fig, axes = plt.subplots(1, 3, figsize=(16, 5))
18 fig.suptitle('Cherenkov Cone Angle | ANNIE water n = 1.34',
19             fontsize=13, fontweight='bold')
20
21 ax = axes[0]
22 ax.plot(beta_grid, theta_deg, color='steelblue', lw=2, label='θ_C(β) analy
23 ax.axvline(BETA_THR, color='black', ls=':', lw=1.5, label=f'β_thr = {BETA
24 ax.set_xlabel('β = v/c'); ax.set_ylabel('Cherenkov cone half-angle (°)')
25 ax.set_title('A θ_C vs β (analytic)')
26 ax.set_xlim(0.74, 1.0); ax.legend(fontsize=9)
27
28 ax = axes[1]
29 ax.hist(mu_cone, bins=60, color=C_MU, alpha=0.7,
30         label=f'μ⁻ mean={np.nanmean(mu_cone):.1f}°')
31 ax.hist(pi_cone, bins=60, color=C_PI, alpha=0.7,
32         label=f'π⁺ mean={np.nanmean(pi_cone):.1f}°')
33 ax.set_xlabel('Cherenkov cone half-angle (°)'); ax.set_ylabel('Events')
34 ax.set_title('B Cone angle distributions (signal sample)'); ax.legend(fo
35
36 ax = axes[2]
37 ax.scatter(mu_beta_sig, mu_cone, s=3, alpha=0.15, color=C_MU, label='μ⁻')
38 ax.scatter(pi_beta_sig, pi_cone, s=3, alpha=0.15, color=C_PI, label='π⁺')
39 ax.plot(beta_grid, theta_deg, color='black', lw=1.5, ls='--', label='analy
40 ax.set_xlabel('β'); ax.set_ylabel('θ_C (°)')
41 ax.set_title('C Cone angle vs β (scatter + theory)'); ax.legend(fontsize=
42
43 plt.tight_layout()
44 # fig.savefig(PNG_DIR / 'plot6_cherenkov_cone_angle.png', dpi=600, bbox_in
45 plt.show()

```



Plot 7 — μ^- hump: stopping vs punch-through (1×3)

In [17]:

```

1  with uproot.open(SHUF_PATH) as f:
2      et_key = sorted([k for k in f.keys() if 'EventTree' in k])[-1]
3      ev_hump = f[et_key].arrays(
4          ['event_id', 'mu_p0', 'mu_cher_total', 'mu_track_len', 'mu_contained']
5  ev_hump['mu_contained'] = ev_hump['mu_contained'].astype(bool)
6  for c in ['mu_p0', 'mu_cher_total', 'mu_track_len']:
7      ev_hump[c] = pd.to_numeric(ev_hump[c], errors='coerce')
8  if ev_hump['mu_track_len'].max() > 600:
9      ev_hump['mu_track_len'] = ev_hump['mu_track_len'] / 10.
10 ev_hump['mu_cher_per_cm'] = np.where(ev_hump['mu_track_len'] > 0,
11                                     ev_hump['mu_cher_total']/ev_hump['mu_
12 stop_mask = np.isfinite(ev_hump['mu_p0']) & np.isfinite(ev_hump['mu_cher
13 punch_mask = np.isfinite(ev_hump['mu_p0']) & np.isfinite(ev_hump['mu_cher
14
15 pmax_mu = ev_hump['mu_p0'].quantile(0.995)
16 p_grid_mu = np.linspace(0.001, pmax_mu*1.05, 300)
17 print('Computing Frank-Tamm for  $\mu^-$  hump plot (~5 s)...')
18 mu_ft_h = frank_tamm_track_averaged(p_grid_mu, M_MU)
19 PTHR_MU_EFF, _ = effective_threshold_momentum(M_MU)
20
21 p_bins_full = np.linspace(0.0, pmax_mu*1.05, 24)
22 p_bins_low = np.linspace(0.0, 0.8, 32)
23
24 def bq(x, y, bins): return binned_quantiles(x, y, bins, min_count=20)
25
26 stop_pc, stop_q, _ = bq(ev_hump.loc[stop_mask, 'mu_p0'], ev_hump.loc[sto
27 punch_pc, punch_q, _ = bq(ev_hump.loc[punch_mask, 'mu_p0'], ev_hump.loc[pu
28 stop_pc_z, stop_q_z, _ = bq(ev_hump.loc[stop_mask, 'mu_p0'], ev_hump.loc
29 punch_pc_z, punch_q_z, _ = bq(ev_hump.loc[punch_mask, 'mu_p0'], ev_hump.loc
30
31 fig, axes = plt.subplots(1, 3, figsize=(18, 6))
32 fig.suptitle('μ- hump verification: stopping vs punch-through | Full 30k s
33               fontsize=13, fontweight='bold')
34
35 for ax, pc_s, q_s, pc_p, q_p, p_g, title in [
36     (axes[0], stop_pc, stop_q, punch_pc, punch_q, p_grid_mu,
37      'A Full momentum range'),
38     (axes[1], stop_pc_z, stop_q_z, punch_pc_z, punch_q_z,
39      p_grid_mu[p_grid_mu < 0.85], 'B Zoom: 0-800 MeV/c'),
40 ]:
41     ax.fill_between(pc_s, q_s[0.16], q_s[0.84], alpha=0.2, color=C_MC)
42     ax.plot(pc_s, q_s[0.50], 'o-', color=C_MC, lw=1.8, ms=4, label='stop
43     ax.fill_between(pc_p, q_p[0.16], q_p[0.84], alpha=0.2, color=C_MU2)
44     ax.plot(pc_p, q_p[0.50], 's-', color=C_MU2, lw=1.8, ms=4, label='punch
45     ax.plot(p_g, frank_tamm_track_averaged(p_g, M_MU), '--', color='black
46     ax.axvline(PTHR_MU_EFF, color='grey', ls=':', lw=1.5, label=f'thresho
47     ax.set_xlabel('μ- initial momentum (GeV/c)'); ax.set_ylabel('dN/dx (p
48     ax.set_title(title); ax.legend(fontsize=9)
49
50 ax = axes[2]
51 ax.hist(ev_hump.loc[stop_mask, 'mu_cher_per_cm'].dropna(), bins=60,
52         color=C_MC, alpha=0.7, label=f'stopping n={stop_mask.sum():,}')
53 ax.hist(ev_hump.loc[punch_mask, 'mu_cher_per_cm'].dropna(), bins=60,
54         color=C_MU2, alpha=0.7, label=f'punch-through n={punch_mask.sum(
55 ax.axvline(FT_LIMIT, color='black', ls=':', lw=2, label=f'F-T β→1 = {FT_L

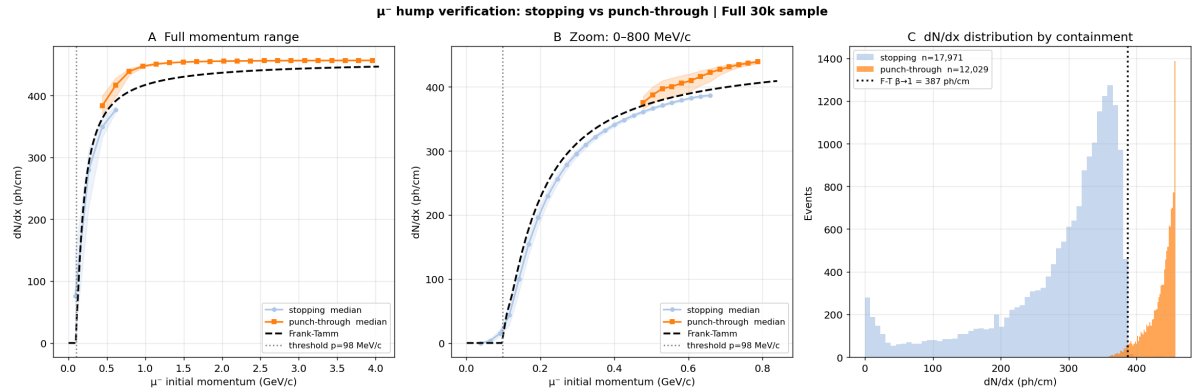
```

```

56 ax.set_xlabel('dN/dx (ph/cm)'); ax.set_ylabel('Events')
57 ax.set_title('C dN/dx distribution by containment'); ax.legend(fontsize=10)
58
59 plt.tight_layout()
60 # fig.savefig(PNG_DIR / 'plot7_muon_hump_stopping_vs_punchthrough.png', dpi=300)
61 plt.show()

```

Computing Frank-Tamm for μ^- hump plot (~5 s)...



Plot 8 — Nuclear effects: π^+ +O inelastic hadronic inventory (3×3)

In [18]:

```

1  # Load inelastic signal events from EventTree
2  with uproot.open(SHUF_PATH) as f:
3      et_key = sorted([k for k in f.keys() if 'EventTree' in k])[-1]
4      ev_inel = f[et_key].arrays(
5          ['event_id', 'mu_p0', 'pi_p0', 'mu_contained', 'pi_final_process',
6           'mu_cher_total', 'pi_cher_total',
7           'pi_x_end', 'pi_y_end', 'pi_z_end', 'pi_KE0'], library='pd')
8      ev_inel['mu_contained'] = ev_inel['mu_contained'].astype(bool)
9      ev_inel['pi_final_process'] = (ev_inel['pi_final_process']
10                                   .astype(str).str.strip().str.rstrip('\x00'))
11     for c in ['mu_p0', 'pi_p0', 'mu_cher_total', 'pi_cher_total',
12              'pi_x_end', 'pi_y_end', 'pi_z_end', 'pi_KE0']:
13         ev_inel[c] = pd.to_numeric(ev_inel[c], errors='coerce')
14
15     sig_inel = ev_inel[
16         (ev_inel['mu_p0'] > MU_P_THRESHOLD) &
17         (~ev_inel['mu_contained']) &
18         ev_inel['pi_final_process'].str.contains('Inelastic', na=False)
19     ].copy()
20     inel_ids = set(sig_inel['event_id'].values)
21     print(f"Inelastic signal events: {len(sig_inel):,}")
22
23     # Load StepTree for inelastic events only
24     ST_COLS_IN = ['event_id', 'particle', 'parent_id', 'track_id',
25                  'from_primary_pi', 'cher_step', 'p_mag', 'KE',
26                  'x', 'y', 'z', 'time', 'process', 'step_num']
27     with uproot.open(SHUF_PATH) as f:
28         st_key = sorted([k for k in f.keys() if 'StepTree' in k])[-1]
29         st_inel_raw = f[st_key].arrays(ST_COLS_IN, library='pd')
30         st_inel_raw['particle'] = st_inel_raw['particle'].astype(str).str.s
31         st_inel_raw['from_primary_pi'] = st_inel_raw['from_primary_pi'].astype(boo
32     for c in ['cher_step', 'p_mag', 'KE']:
33         st_inel_raw[c] = pd.to_numeric(st_inel_raw[c], errors='coerce').fillna
34
35     st_ev = st_inel_raw[st_inel_raw['event_id'].isin(inel_ids)].copy()
36     is_prim_pi = (st_ev['particle'] == 'pi+') & (st_ev['parent_id'] == 0)
37     is_had_sec = st_ev['from_primary_pi'] & ~is_prim_pi
38     first_steps = (st_ev[is_had_sec]
39                   .sort_values('step_num')
40                   .groupby(['event_id', 'track_id']).first().reset_index())
41
42     mass_map = {'pi+': 0.13957, 'pi-': 0.13957, 'proton': 0.93827, 'neutron': 0.9395
43                'deuteron': 1.87561, 'triton': 2.80892, 'alpha': 3.72742,
44                'mu+': 0.10566, 'mu-': 0.10566, 'e+': 0.000511, 'e-': 0.000511}
45     def p_thr(mass): return mass / np.sqrt(N_WATER**2 - 1)
46     first_steps['mass_GeV'] = first_steps['particle'].map(mass_map).fillna(1.
47     first_steps['p_thr'] = first_steps['mass_GeV'].apply(p_thr)
48     first_steps['above_thr'] = first_steps['p_mag'] > first_steps['p_thr']
49
50     # Per-event Cherenkov from secondaries
51     def cher_by_species(pname):
52         m = is_had_sec & (st_ev['particle'] == pname)
53         return st_ev[m].groupby('event_id')['cher_step'].sum().rename(f'cher_
54     sec_cher = is_had_sec
55     sec_cher_sum = st_ev[is_had_sec].groupby('event_id')['cher_step'].sum().r

```

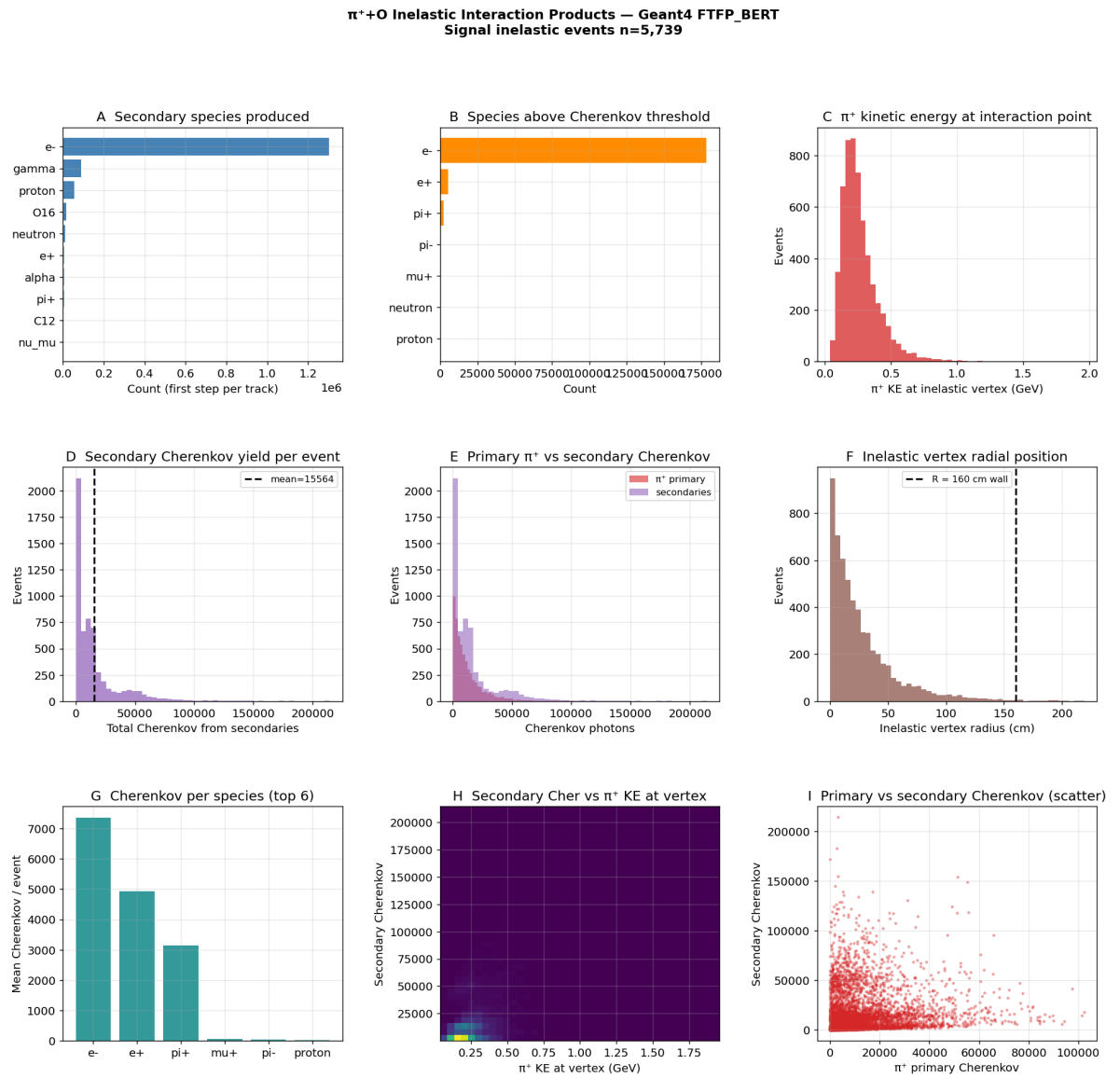
```

56 species_to_track = ['pi+', 'pi-', 'proton', 'e-', 'e+', 'mu+', 'deuteron', 'alph
57 cher_cols = [cher_by_species(s) for s in species_to_track]
58 sig_inel = (sig_inel.set_index('event_id')
59             .join(sec_cher_sum, how='left')
60             .join(pd.concat(cher_cols, axis=1), how='left')
61             .reset_index())
62 for col in ['sec_cher'] + [f'cher_{s}' for s in species_to_track]:
63     if col in sig_inel.columns:
64         sig_inel[col] = sig_inel[col].fillna(0.)
65
66 sig_inel['pi_r_end'] = np.sqrt(
67     sig_inel['pi_x_end']**2 + sig_inel['pi_z_end']**2) / 10. # mm→cm
68
69 fig = plt.figure(figsize=(18, 16))
70 gs = gridspec.GridSpec(3, 3, figure=fig, hspace=0.45, wspace=0.35)
71 fig.suptitle(f'π++0 Inelastic Interaction Products – Geant4 FTFP_BERT\n'
72             f'Signal inelastic events n={len(sig_inel):,}',
73             fontsize=13, fontweight='bold')
74
75 species_counts = first_steps['particle'].value_counts()
76 above_thr_cnt = first_steps[first_steps['above_thr']][['particle']].value_
77
78 ax = fig.add_subplot(gs[0, 0])
79 top_sp = species_counts.head(10)
80 ax.barh(top_sp.index[::-1], top_sp.values[::-1], color='steelblue')
81 ax.set_xlabel('Count (first step per track)'); ax.set_title('A Secondary
82
83 ax = fig.add_subplot(gs[0, 1])
84 top_abv = above_thr_cnt.head(8)
85 ax.barh(top_abv.index[::-1], top_abv.values[::-1], color='darkorange')
86 ax.set_xlabel('Count'); ax.set_title('B Species above Cherenkov threshol
87
88 ax = fig.add_subplot(gs[0, 2])
89 ax.hist(sig_inel['pi_KE0'], bins=50, color=C_PI, alpha=0.75)
90 ax.set_xlabel('π+ KE at inelastic vertex (GeV)'); ax.set_ylabel('Events')
91 ax.set_title('C π+ kinetic energy at interaction point')
92
93 ax = fig.add_subplot(gs[1, 0])
94 ax.hist(sig_inel['sec_cher'].dropna(), bins=50, color=C_INEL, alpha=0.75)
95 ax.axvline(sig_inel['sec_cher'].mean(), ls='--', lw=1.8, color='black',
96            label=f'mean={sig_inel["sec_cher"].mean():.0f}')
97 ax.set_xlabel('Total Cherenkov from secondaries'); ax.set_ylabel('Events')
98 ax.set_title('D Secondary Cherenkov yield per event'); ax.legend(fontsiz
99
100 ax = fig.add_subplot(gs[1, 1])
101 ax.hist(sig_inel['pi_cher_total'], bins=50, color=C_PI, alpha=0.6, label=
102 ax.hist(sig_inel['sec_cher'].dropna(), bins=50, color=C_INEL, alpha=0.6,
103 ax.set_xlabel('Cherenkov photons'); ax.set_ylabel('Events')
104 ax.set_title('E Primary π+ vs secondary Cherenkov'); ax.legend(fontsize=
105
106 ax = fig.add_subplot(gs[1, 2])
107 ax.hist(sig_inel['pi_r_end'].dropna(), bins=50, color=C_OTHER, alpha=0.75
108 ax.axvline(160., color='black', ls='--', lw=1.8, label='R = 160 cm wall')
109 ax.set_xlabel('Inelastic vertex radius (cm)'); ax.set_ylabel('Events')
110 ax.set_title('F Inelastic vertex radial position'); ax.legend(fontsize=9
111

```

```
112 cher_species_means = {s: sig_inel[f'cher_{s}'].mean() for s in species_to
113 top_cher = sorted(cher_species_means.items(), key=lambda x: x[1], reverse
114 ax = fig.add_subplot(gs[2, 0])
115 labels_c = [t[0] for t in top_cher]
116 vals_c = [t[1] for t in top_cher]
117 ax.bar(labels_c, vals_c, color='teal', alpha=0.8)
118 ax.set_ylabel('Mean Cherenkov / event'); ax.set_title('G Cherenkov per s
119
120 ax = fig.add_subplot(gs[2, 1])
121 ax.hist2d(sig_inel['pi_KE0'], sig_inel['sec_cher'].fillna(0),
122           bins=40, cmap='viridis')
123 ax.set_xlabel('π+ KE at vertex (GeV)'); ax.set_ylabel('Secondary Cherenko
124 ax.set_title('H Secondary Cher vs π+ KE at vertex')
125
126 ax = fig.add_subplot(gs[2, 2])
127 ax.scatter(sig_inel['pi_cher_total'], sig_inel['sec_cher'].fillna(0),
128           s=4, alpha=0.3, color=C_PI)
129 ax.set_xlabel('π+ primary Cherenkov'); ax.set_ylabel('Secondary Cherenkov
130 ax.set_title('I Primary vs secondary Cherenkov (scatter)')
131
132 # fig.savefig(PNG_DIR / 'plot8_nuclear_inelastic_inventory.png', dpi=600,
133 plt.show()
```

Inelastic signal events: 5,739



Plot 9 — Radioactive isotope inventory from π^+ +O spallation

In [19]:

```

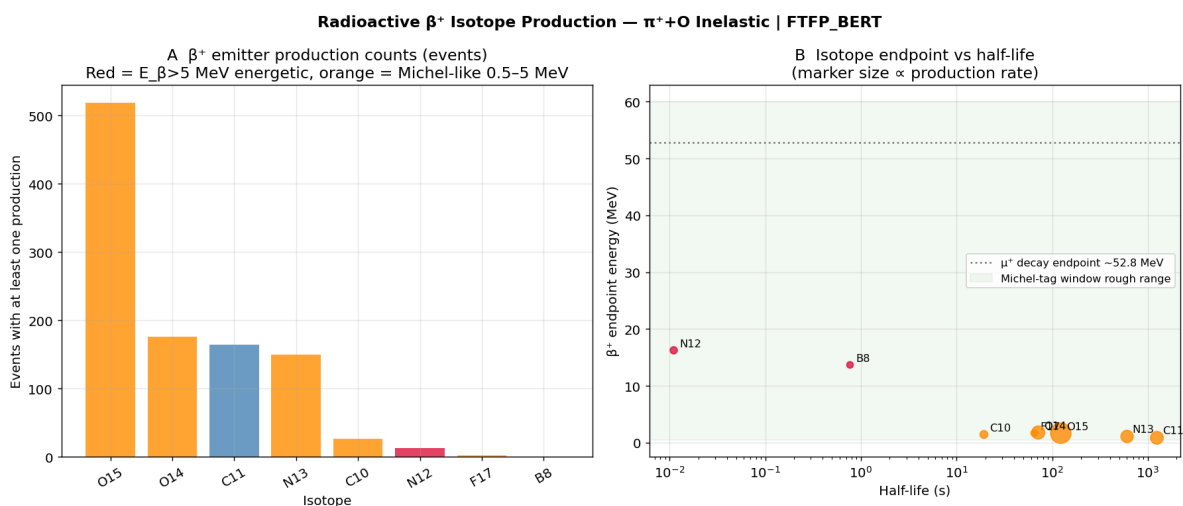
1  BETA_PLUS_ISOTOPES = {
2      'N12': (0.011, 16.32, 'β+'),
3      'B8': (0.770, 13.70, 'β+'),
4      'C10': (19.3, 1.47, 'β+'),
5      'O15': (122., 1.72, 'β+'),
6      'N13': (597., 1.19, 'β+'),
7      'C11': (1224., 0.96, 'β+'),
8      'F17': (64.5, 1.74, 'β+'),
9      'O14': (70.6, 1.81, 'β+'),
10 }
11
12 # Production inventory from StepTree (search for these nuclei in inelastic
13 with uproot.open(SHUF_PATH) as f:
14     st_key = sorted([k for k in f.keys() if 'StepTree' in k])[-1]
15     st_iso = f[st_key].arrays(
16         ['event_id', 'particle', 'parent_id', 'from_primary_pi', 'p_mag', 'KE'],
17         library='pd')
18     st_iso['particle'] = st_iso['particle'].astype(str).strip().str
19     st_iso['from_primary_pi'] = st_iso['from_primary_pi'].astype(bool)
20
21     st_iso_ev = st_iso[st_iso['event_id'].isin(inel_ids)].copy()
22     iso_names = list(BETA_PLUS_ISOTOPES.keys())
23     iso_counts = {}
24     for iso in iso_names:
25         mask = (st_iso_ev['particle'] == iso) & (st_iso_ev['from_primary_pi'])
26         iso_counts[iso] = st_iso_ev[mask]['event_id'].nunique()
27
28     fig, axes = plt.subplots(1, 2, figsize=(14, 6))
29     fig.suptitle('Radioactive β+ Isotope Production – π++0 Inelastic | FTFP_B
30                 fontsize=13, fontweight='bold')
31
32     # Panel A: production counts
33     ax = axes[0]
34     sorted_iso = sorted(iso_counts.items(), key=lambda x: x[1], reverse=True)
35     labels_iso = [k for k, v in sorted_iso]
36     vals_iso = [v for k, v in sorted_iso]
37     colors_iso = ['crimson' if BETA_PLUS_ISOTOPES[iso][1] > 5 else
38                  'darkorange' if BETA_PLUS_ISOTOPES[iso][1] > 1 else 'steelbl
39                  for iso in labels_iso]
40     ax.bar(labels_iso, vals_iso, color=colors_iso, alpha=0.8)
41     ax.set_xlabel('Isotope'); ax.set_ylabel('Events with at least one product
42     ax.set_title('A β+ emitter production counts (events)\n'
43                 'Red = Eβ>5 MeV energetic, orange = Michel-like 0.5–5 MeV')
44     ax.tick_params(axis='x', rotation=30)
45
46     # Panel B: β+ endpoint energy vs half-life scatter
47     ax = axes[1]
48     for iso, (t_half, E_end, _) in BETA_PLUS_ISOTOPES.items():
49         color = ('crimson' if E_end > 5 else
50                'darkorange' if 0.5 < E_end < 5 else
51                'steelblue')
52         ax.scatter(t_half, E_end, s=iso_counts.get(iso, 0)*0.5+30,
53                   color=color, alpha=0.8, zorder=3)
54         ax.annotate(iso, (t_half, E_end), fontsize=9,
55                     xytext=(5, 3), textcoords='offset points')

```

```

56 ax.axhline(52.8, color='grey', ls=':', lw=1.5, label='μ+ decay endpoint ~52.8 MeV')
57 ax.axhspan(0.5, 60., alpha=0.05, color='green', label='Michel-tag window')
58 ax.set_xscale('log')
59 ax.set_xlabel('Half-life (s)'); ax.set_ylabel('β+ endpoint energy (MeV)')
60 ax.set_title('B Isotope endpoint vs half-life\n(marker size ∝ production rate)')
61 ax.legend(fontsize=9)
62
63 plt.tight_layout()
64 # fig.savefig(PNG_DIR / 'plot9_isotope_inventory.png', dpi=600, bbox_inches='tight')
65 plt.show()
66
67 print('\nβ+ isotope production summary:')
68 print(f'{"Isotope":8s} {"t½":>10s} {"E_β (MeV)":>12s} {"Events":>8s} {"Concern":>15s}')
69 print('-'*55)
70 for iso, (t_half, E_end, _) in sorted(BETA_PLUS_ISOTOPES.items(), key=lambda x: x[1][0]):
71     t_str = f'{t_half*1000:.0f} ms' if t_half<1 else f'{t_half:.1f} s'
72     t_str = f'{t_half/60:.1f} min' if t_half<3600 else f'{t_half/3600:.1f} yr'
73     concern = ('⚠ ENERGETIC' if E_end > 5 else
74               '⚠ MICHEL-LIKE' if 0.5 < E_end < 5 else 'low energy')
75     print(f' {iso:6s} {t_str:>10s} {E_end:>8.2f} MeV {iso_counts.get(iso, 0):>8s} {concern}')

```



β⁺ isotope production summary:

Isotope	t½	E _β (MeV)	Events	Concern
N12	11 ms	16.32 MeV	13	⚠ ENERGETIC
B8	770 ms	13.70 MeV	0	⚠ ENERGETIC
C10	19.3 s	1.47 MeV	26	⚠ MICHEL-LIKE
F17	1.1 min	1.74 MeV	2	⚠ MICHEL-LIKE
O14	1.2 min	1.81 MeV	176	⚠ MICHEL-LIKE
O15	2.0 min	1.72 MeV	519	⚠ MICHEL-LIKE
N13	9.9 min	1.19 MeV	150	⚠ MICHEL-LIKE
C11	20.4 min	0.96 MeV	164	⚠ MICHEL-LIKE

Plot 10 — Decay timing: full waveform prompt + delayed (log-log)

In [20]:

```

1  # Accumulate Log-spaced PMT hit-time histograms from StepTree
2  LOG_BINS = np.logspace(np.log10(1), np.log10(20000), 200) # 1 ns → 20 μs
3  LOG_CENTS = np.sqrt(LOG_BINS[:-1] * LOG_BINS[1:])
4  LOG_WIDTHS = np.diff(LOG_BINS)
5
6  acc_log = {k: np.zeros(len(LOG_CENTS)) for k in ['mu', 'pi', 'smu', 'michel']}
7  n_log = {k: 0 for k in acc_log}
8
9  st_cols_decay = ['event_id', 'particle', 'parent_id', 'track_id',
10                  'x', 'y', 'z', 'time', 'cher_step', 'from_primary_pi']
11
12  # Need mu+ track map first
13  print('Pass 1: Building mu+ track map...')
14  mu_plus_map = {}
15  with uproot.open(SHUF_PATH) as f:
16      st_key = sorted([k for k in f.keys() if 'StepTree' in k])[-1]
17      for chunk in f[st_key].iterate(
18          ['event_id', 'particle', 'track_id', 'from_primary_pi'],
19          step_size=500_000, library='pd'):
20          chunk['particle'] = chunk['particle'].astype(str).str.strip()
21          chunk['from_primary_pi'] = chunk['from_primary_pi'].astype(bool)
22          mup = chunk[(chunk['particle']=='mu+') & chunk['from_primary_pi']]
23                  & chunk['event_id'].isin(decay_ids)]
24          for eid, grp in mup.groupby('event_id'):
25              if eid not in mu_plus_map: mu_plus_map[eid] = set()
26              mu_plus_map[eid].update(grp['track_id'].tolist())
27  print(f' mu+ tracks in {len(mu_plus_map):,} decay events')
28
29  print('Pass 2: Accumulating log-spaced hit times...')
30  with uproot.open(SHUF_PATH) as f:
31      st_key = sorted([k for k in f.keys() if 'StepTree' in k])[-1]
32      for chunk in f[st_key].iterate(st_cols_decay, step_size=500_000, library='pd'):
33          chunk['particle'] = chunk['particle'].astype(str).str.strip()
34          chunk['from_primary_pi'] = chunk['from_primary_pi'].astype(bool)
35          chunk = chunk[(chunk['event_id'].isin(decay_ids)) & (chunk['cher_step'] > 0)]
36          if len(chunk) == 0: continue
37          d_wall = tof_to_cylinder(chunk['x'].values, chunk['y'].values, R_WATER)
38          chunk = chunk.copy()
39          chunk['t_hit'] = chunk['time'] + d_wall / C_WATER
40          chunk['cher_det'] = chunk['cher_step'] * DET_EFF
41          is_mu = (chunk['particle']=='mu-') & (chunk['parent_id']==0)
42          is_pi = (chunk['particle']=='pi+') & chunk['from_primary_pi']
43          is_smu = (chunk['particle']=='mu+') & chunk['from_primary_pi']
44          ep = chunk[(chunk['particle']=='e+') & chunk['from_primary_pi']]
45          if len(ep):
46              m_idx = np.array([pid in mu_plus_map.get(eid, set())
47                              for pid, eid in zip(ep['parent_id'].values, ep['event_id'].values)])
48              is_michel_idx = ep.index[m_idx]
49          else:
50              is_michel_idx = pd.Index([])
51          for key, sub in [(('mu', chunk[is_mu]), ('pi', chunk[is_pi])),
52                          (('smu', chunk[is_smu]),
53                           ('michel', chunk.loc[is_michel_idx] if len(is_michel_idx) > 0 else None))]:
54              if len(sub) == 0: continue
55              t = np.clip(sub['t_hit'].values, LOG_BINS[0], None)

```

```

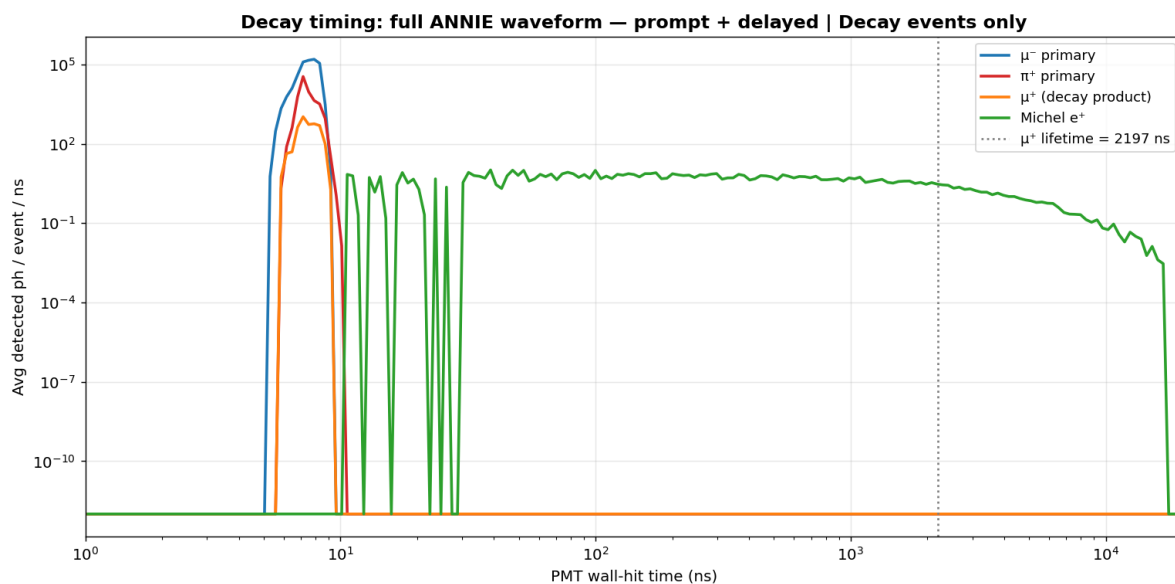
56         w = sub['cher_det'].values
57         acc_log[key] += np.histogram(t, bins=LOG_BINS, weights=w)[0]
58         n_log[key] += 1
59
60     n_ev = max(n_log['mu'], 1)
61     rate_mu = acc_log['mu'] / (n_ev * LOG_WIDTHS)
62     rate_pi = acc_log['pi'] / (n_ev * LOG_WIDTHS)
63     rate_smu = acc_log['smu'] / (n_ev * LOG_WIDTHS)
64     rate_michel = acc_log['michel'] / (n_ev * LOG_WIDTHS)
65
66     fig, ax = plt.subplots(figsize=(12, 6))
67     ax.loglog(LOG_CENTS, np.where(rate_mu>0, rate_mu, 1e-12), color=C_MU)
68     ax.loglog(LOG_CENTS, np.where(rate_pi>0, rate_pi, 1e-12), color=C_PI)
69     ax.loglog(LOG_CENTS, np.where(rate_smu>0, rate_smu, 1e-12), color=C_SMU)
70     ax.loglog(LOG_CENTS, np.where(rate_michel>0, rate_michel, 1e-12), color=C_DM)
71     ax.axvline(2197., color='grey', ls=':', lw=1.5, label='μ+ lifetime = 2197 ns')
72     ax.set_xlabel('wall-hit time (ns)')
73     ax.set_ylabel('Avg detected ph / event / ns')
74     ax.set_title('Decay timing: full ANNIE waveform – prompt + delayed | Decay')
75     ax.fontweight='bold')
76     ax.legend(fontsize=10)
77     ax.set_xlim(1, 20000)
78     plt.tight_layout()
79     # fig.savefig(PNG_DIR / 'plot10_decay_timing_loglog.png', dpi=600, bbox_inches='tight')
80     plt.show()

```

Pass 1: Building mu+ track map...

mu+ tracks in 6,272 decay events

Pass 2: Accumulating log-spaced hit times...



Plot 11 — Prompt shoulder: pion vs muon timing (0–60 ns)

In [23]:

```

1  T_PROMPT    = np.linspace(0, 60, 241)
2  T_PROMPT_C  = (T_PROMPT[:-1] + T_PROMPT[1:]) / 2
3  sum_mu_p    = np.zeros(len(T_PROMPT_C))
4  sum_pi_p    = np.zeros(len(T_PROMPT_C))
5  n_mu_ev     = 0; n_pi_ev = 0
6  pion_outlives_ids = set(np.int32(list(
7      ev_sig[ev_sig['pi_time_end'] > ev_sig['mu_water_exit_ns']].index)))
8  sum_mu_out  = np.zeros(len(T_PROMPT_C))
9  sum_pi_out  = np.zeros(len(T_PROMPT_C))
10 n_mu_out    = 0
11 n_pi_out    = 0
12
13 st_cols_sh = ['event_id', 'particle', 'parent_id', 'x', 'y', 'z', 'time', 'cher_
14 print(f"Pion-outlives-muon events: {len(pion_outlives_ids):,} "
15       f"({len(pion_outlives_ids)/len(ev_sig)*100:.1f}%)")
16
17 with uproot.open(SHUF_PATH) as f:
18     st_key = sorted([k for k in f.keys() if 'StepTree' in k])[-1]
19     for i, chunk in enumerate(f[st_key].iterate(st_cols_sh, step_size=500,
20         chunk['particle'] = chunk['particle'].astype(str).str.strip
21         chunk['from_primary_pi'] = chunk['from_primary_pi'].astype(bool)
22         mask = (
23             chunk['event_id'].isin(sig_ids) & (chunk['cher_step'] > 0) &
24             (((chunk['particle']=='mu-') & (chunk['parent_id']==0)) |
25             (chunk['particle']=='pi+'))
26         )
27         chunk = chunk[mask].copy()
28         if len(chunk) == 0: continue
29         chunk = chunk.merge(mu_exit_lookup, left_on='event_id', right_index=
30         d_wall = tof_to_cylinder(chunk['x'].values, chunk['y'].values, chunk['z'].values)
31         chunk['t_hit'] = chunk['time'] + d_wall / C_WATER
32         chunk['cher_det'] = chunk['cher_step'] * DET_EFF
33         is_mu = (chunk['particle']=='mu-') & (chunk['parent_id']==0)
34         is_pi = (chunk['particle']=='pi+')
35         for eid, grp in chunk[is_mu].groupby('event_id'):
36             sum_mu_p += np.histogram(grp['t_hit'], bins=T_PROMPT, weights=grp['cher_det'])
37         for eid, grp in chunk[is_pi].groupby('event_id'):
38             sum_pi_p += np.histogram(grp['t_hit'], bins=T_PROMPT, weights=grp['cher_det'])
39         out_mu = chunk[is_mu & chunk['event_id'].isin(pion_outlives_ids)]
40         out_pi = chunk[is_pi & chunk['event_id'].isin(pion_outlives_ids)]
41         for eid, grp in out_mu.groupby('event_id'):
42             sum_mu_out += np.histogram(grp['t_hit'], bins=T_PROMPT, weights=grp['cher_det'])
43         for eid, grp in out_pi.groupby('event_id'):
44             sum_pi_out += np.histogram(grp['t_hit'], bins=T_PROMPT, weights=grp['cher_det'])
45         if (i+1)%10==0: print(f'  chunk {i+1}')
46
47 avg_mu_p    = sum_mu_p / max(n_mu_ev, 1)
48 avg_pi_p    = sum_pi_p / max(n_pi_ev, 1)
49 avg_mu_out  = sum_mu_out / max(n_mu_out, 1)
50 avg_pi_out  = sum_pi_out / max(n_pi_out, 1)
51
52 fig, axes = plt.subplots(1, 2, figsize=(14, 5))
53 fig.suptitle('Prompt shoulder:  $\mu^-$  vs  $\pi^+$  timing | 0-60 ns | Signal sample',
54             fontsize=13, fontweight='bold')
55

```

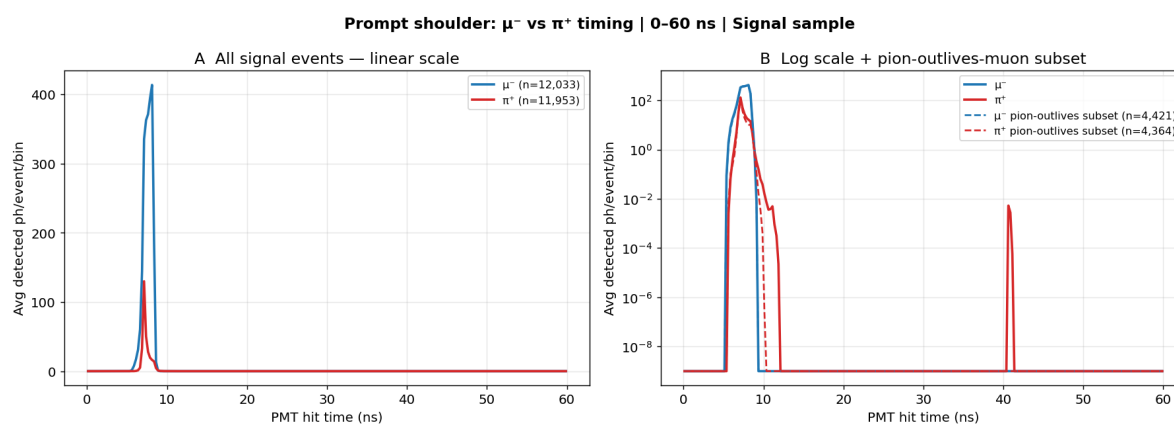
```

56 ax = axes[0]
57 ax.plot(T_PROMPT_C, avg_mu_p, color=C_MU, lw=2, label=f'μ- (n={n_mu_ev:,})')
58 ax.plot(T_PROMPT_C, avg_pi_p, color=C_PI, lw=2, label=f'π+ (n={n_pi_ev:,})')
59 ax.set_xlabel('PMT hit time (ns)'); ax.set_ylabel('Avg detected ph/event/t')
60 ax.set_title('A All signal events — linear scale'); ax.legend(fontsize=9)
61
62 ax = axes[1]
63 ax.semilogy(T_PROMPT_C, np.where(avg_mu_p>0, avg_mu_p, 1e-9), color=C_MU,
64 ax.semilogy(T_PROMPT_C, np.where(avg_pi_p>0, avg_pi_p, 1e-9), color=C_PI,
65 ax.semilogy(T_PROMPT_C, np.where(avg_mu_out>0, avg_mu_out, 1e-9), color=C_MU,
66             label=f'μ- pion-outlives subset (n={n_mu_out:,})')
67 ax.semilogy(T_PROMPT_C, np.where(avg_pi_out>0, avg_pi_out, 1e-9), color=C_PI,
68             label=f'π+ pion-outlives subset (n={n_pi_out:,})')
69 ax.set_xlabel('PMT hit time (ns)'); ax.set_ylabel('Avg detected ph/event/t')
70 ax.set_title('B Log scale + pion-outlives-muon subset'); ax.legend(fonts:
71
72 plt.tight_layout()
73 # fig.savefig(PNG_DIR / 'plot11_prompt_shoulder.png', dpi=600, bbox_inches=
74 plt.show()

```

Pion-outlives-muon events: 4,418 (36.7%)

chunk 10
 chunk 20
 chunk 30
 chunk 40
 chunk 50
 chunk 60



Plot 12 — High-resolution prompt zoom (4–12 ns, event 23630 removed)

In [22]:

```

1  OUTLIER_EIDS = {LATE_OUTLIER_EID}
2  ev_sig_plot = ev_sig_no23630.copy()
3  if 'event_id' not in ev_sig_plot.columns:
4      ev_sig_plot['event_id'] = ev_sig_plot.index
5  ev_sig_plot['event_id'] = pd.to_numeric(ev_sig_plot['event_id'], errors='coerce')
6  ev_sig_plot = ev_sig_plot.dropna(subset=['event_id']).copy()
7  ev_sig_plot['event_id'] = ev_sig_plot['event_id'].astype(np.int32)
8  ev_sig_plot = ev_sig_plot.set_index('event_id', drop=False)
9  print(f'Plotting sample: {len(ev_sig_plot):,} events')
10
11  T_ZOOM = np.arange(4.0, 12.0 + 0.1, 0.1)
12  T_ZOOM_C = 0.5 * (T_ZOOM[:-1] + T_ZOOM[1:])
13  sum_mu_z = np.zeros(len(T_ZOOM_C))
14  sum_pi_ch = np.zeros(len(T_ZOOM_C)) # chain-based
15  sum_pi_st = np.zeros(len(T_ZOOM_C)) # strict primary
16  sig_ids_z = set(np.int32(ev_sig_plot.index.tolist()))
17  pi_tid_lk = ev_sig_plot['pi_track_id'].to_dict()
18  n_mu_z = 0; n_pi_ch_z = 0; n_pi_st_z = 0
19
20  zoom_cols = ['event_id', 'particle', 'parent_id', 'track_id', 'x', 'y', 'z', 'time']
21  with uproot.open(SHUF_PATH) as f:
22      st_key = sorted([k for k in f.keys() if 'StepTree' in k])[-1]
23      for i, chunk in enumerate(f[st_key].iterate(zoom_cols, step_size=500)):
24          chunk['particle'] = chunk['particle'].astype(str).str.strip()
25          chunk['from_primary_pi'] = chunk['from_primary_pi'].astype(bool)
26          chunk = chunk[(chunk['event_id'].isin(sig_ids_z)) & (chunk['charge'] != 0)]
27          if len(chunk)==0: continue
28          d_wall = tof_to_cylinder(chunk['x'].values, chunk['y'].values, chunk['z'].values)
29          chunk['t_hit'] = chunk['time'] + d_wall / C_WATER
30          chunk['cher_det'] = chunk['cher_step'] * DET_EFF
31          mu = chunk[(chunk['particle']=='mu-') & (chunk['parent_id']!=0)]
32          pi_ch = chunk[(chunk['particle']=='pi+') & (chunk['from_primary_pi']==True)]
33          pi_st = chunk[(chunk['particle']=='pi+') & (chunk['from_primary_pi']==False)]
34          if len(pi_st):
35              pi_st['primary_tid'] = pi_st['event_id'].map(pi_tid_lk)
36              pi_st = pi_st[pi_st['track_id']==pi_st['primary_tid']]
37          for _, grp in mu.groupby('event_id'):
38              sum_mu_z += np.histogram(grp['t_hit'], bins=T_ZOOM, weights=grp['t_hit'])[0]
39          for _, grp in pi_ch.groupby('event_id'):
40              sum_pi_ch += np.histogram(grp['t_hit'], bins=T_ZOOM, weights=grp['t_hit'])[0]
41          for _, grp in pi_st.groupby('event_id'):
42              sum_pi_st += np.histogram(grp['t_hit'], bins=T_ZOOM, weights=grp['t_hit'])[0]
43          if (i+1)%10==0: print(f' chunk {i+1}')
44
45  avg_mu_z = sum_mu_z / max(n_mu_z, 1)
46  avg_pi_ch = sum_pi_ch / max(n_pi_ch_z, 1)
47  avg_pi_st = sum_pi_st / max(n_pi_st_z, 1)
48
49  def unit_area(y):
50      s = np.nansum(y); return y/s if s>0 else y
51
52  fig, axes = plt.subplots(1, 2, figsize=(14, 5))
53  fig.suptitle('High-res prompt zoom (4-12 ns) | Event 23630 removed | Signal')
54  fig.text(0.5, 0.05, 'Prompt', fontweight='bold', fontstyle='italic', fontfamily='serif')
55

```

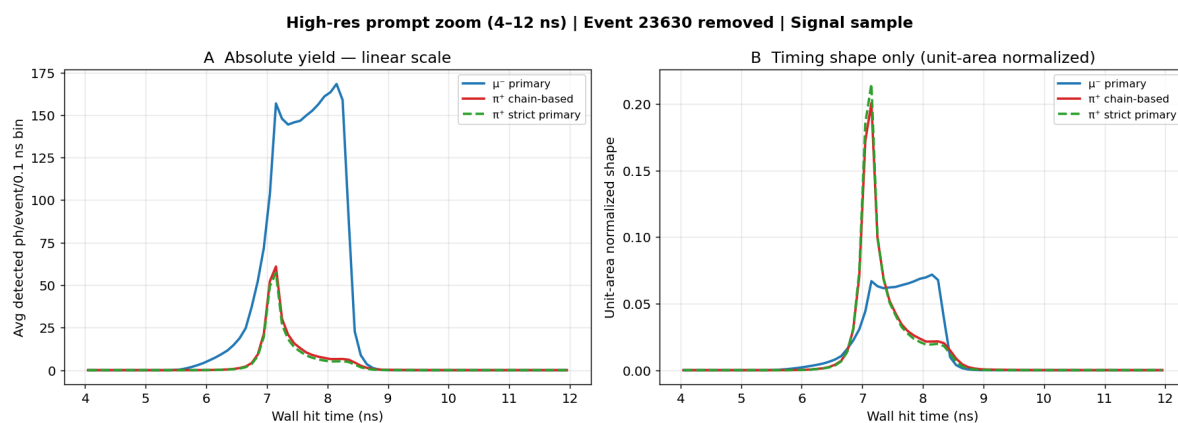
```

56 ax = axes[0]
57 ax.plot(T_ZOOM_C, avg_mu_z, label='μ- primary', lw=2, color=C_MU)
58 ax.plot(T_ZOOM_C, avg_pi_ch, label='π+ chain-based', lw=2, color=C_PI)
59 ax.plot(T_ZOOM_C, avg_pi_st, label='π+ strict primary', lw=2, color=C_DECAY)
60 ax.set_xlabel('Wall hit time (ns)'); ax.set_ylabel('Avg detected ph/event,')
61 ax.set_title('A Absolute yield — linear scale'); ax.legend(fontsize=9)
62
63 ax = axes[1]
64 ax.plot(T_ZOOM_C, unit_area(avg_mu_z), lw=2, color=C_MU, label='μ- pri
65 ax.plot(T_ZOOM_C, unit_area(avg_pi_ch), lw=2, color=C_PI, label='π+ cha
66 ax.plot(T_ZOOM_C, unit_area(avg_pi_st), lw=2, color=C_DECAY, ls='--', lab
67 ax.set_xlabel('Wall hit time (ns)'); ax.set_ylabel('Unit-area normalized s
68 ax.set_title('B Timing shape only (unit-area normalized)'); ax.legend(fo
69
70 plt.tight_layout()
71 # fig.savefig(PNG_DIR / 'plot12_prompt_zoom_hires.png', dpi=600, bbox_inches=
72 plt.show()
73
74 print('='*60)
75 print('HIGH-RES PROMPT ZOOM SUMMARY')
76 print('='*60)
77 print(f'μ- peak time : {T_ZOOM_C[np.nanargmax(avg_mu_z)]:.2f}')
78 print(f'π+ chain-based peak time : {T_ZOOM_C[np.nanargmax(avg_pi_ch)]:.2f}')
79 print(f'π+ strict primary peak time: {T_ZOOM_C[np.nanargmax(avg_pi_st)]:.2f}')
80 print(f'μ- integrated (4-12 ns) : {np.nansum(avg_mu_z):.3f}')
81 print(f'π+ chain-based (4-12 ns) : {np.nansum(avg_pi_ch):.3f}')
82 print(f'π+ strict primary (4-12 ns): {np.nansum(avg_pi_st):.3f}')

```

Plotting sample: 12,028 events

chunk 10
 chunk 20
 chunk 30
 chunk 40
 chunk 50
 chunk 60



```
=====
HIGH-RES PROMPT ZOOM SUMMARY
=====
μ- peak time           : 8.15 ns
π+ chain-based peak time : 7.15 ns
π+ strict primary peak time: 7.15 ns
μ- integrated (4-12 ns)   : 2345.023
π+ chain-based (4-12 ns)  : 303.734
π+ strict primary (4-12 ns): 266.053
```

In []:

1